

Crash Pack: Backend and Systems for the AI Platform

This volume is a high-density reference for the backend or systems engineer who works next to AI models rather than inside them. The research teams train and tune the models. You own the serving fleet, the queues, the APIs, the storage, and the dashboards that page you. This pack compresses the platform half of the curriculum into five parts. Decision rules you apply under time pressure. Numbers and formulas worth memorizing. Traps that have caused real outages. 26 self-test drills with worked answers. A one-page cheat sheet.

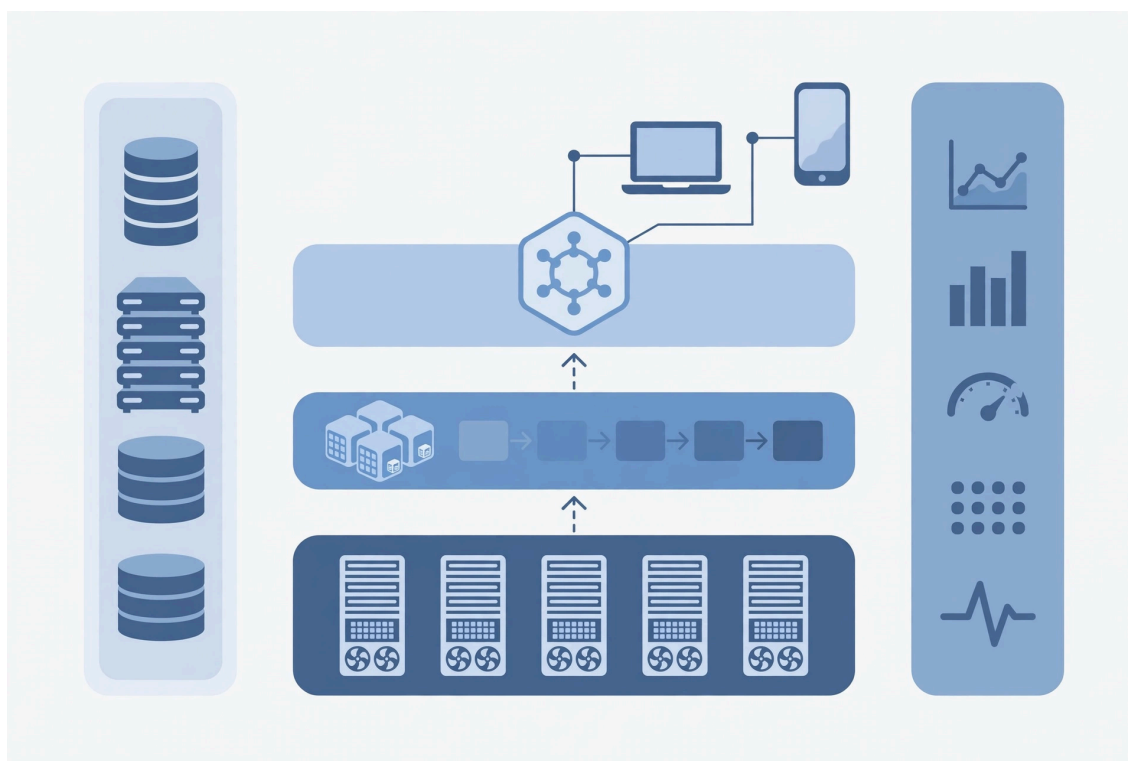


Figure C.1. The territory. Models sit at the center, but a working AI product is mostly platform: compute, serving, queues, storage, APIs, and observability. This pack covers the platform decisions.

Key takeaways

- Six decision rules cover most platform calls: quorum sizing, consistency selection, queue vs stream, batch vs micro-batch, cache sizing, GPU fleet sizing.
- Seven formulas carry the math: Little's Law, M/M/1 waits, HNSW parameters, KV-cache bytes, 2 FLOP per token, burn rate, idempotency patterns.
- Five traps cause most outages: split-brain without fencing, exactly-once illusions, cache stampedes, p99 confusion, silent queue backlogs.
- I executed every code sample in this volume during writing. Expected outputs sit in comments next to the prints that produce them.

How to use this volume

Read Part 1 and Part 2 straight through once. Work every drill in Part 4 with the answers covered, out loud, until the numbers come without pausing. Keep Part 5 open during design work. Revisit Part 3 after your first on-call rotation: the traps read differently after your first page.

Part 1. Decision rules

The chapters ahead follow the platform in order: decisions first, then the numbers behind them, then the traps, then drills. This part is the decisions: six rules, each naming its inputs, its formula or threshold, and what changes the answer. A decision rule is a test you can run in under a minute that picks the design.

1.1 Quorum sizing**Step 1: one copy is a single point of failure.**

A machine dies and its data dies with it. So keep the data on **N** machines. A write counts once **W** of them acknowledge it. A read asks **R** of them. Choosing **W** and **R** is the whole game.

Step 2: the naive answers both fail.

Write to 1, read from 1: fast, and a read can miss a write that landed on another replica. Write to all 3, read from 1: consistent, but one slow or dead replica stalls every write. The middle ground is quorums.

Step 3: the overlap rule.

Pick **W** and **R** so that $W + R > N$. Then every read overlaps every write: they share at least one replica, and that replica carries the newest write. With $N = 3$, $W = 2$, $R = 2$, any read touches a replica the write touched, because $2 + 2 > 3$. Drop to $W = 1$, $R = 1$

and a write on replica A is invisible to a read of B and C.

QUORUM OVERLAP (N=3, W=2, R=2)

Write touches: [A] [B] .

Read touches: . [B] [C]

^

B is in both sets, so the read sees the write.

$2 + 2 > 3$ guarantees overlap for any choice of 2.

How to read this diagram

1. Each row is a set of replicas. Brackets mark the replicas the operation contacts.
2. The caret marks replica B, which sits in both sets. That shared replica carries the newest write stamp.
3. Because any 2 of 3 replicas share at least one member, no read can miss a completed write. The arithmetic does the guaranteeing, not luck.

Step 4: the production picture: add the failure budget.

The second half of the rule is availability. To survive f dead replicas you need $W \leq N - f$ and $R \leq N - f$. The standard picks:

$W + R > N$ (correctness: every read overlaps every write)

$W \leq N - f, R \leq N - f$ (availability: survive f dead replicas)

where: W is the write quorum (acknowledgments needed); R is the read quorum (replicas asked); N is the replica count; f is the failures you must survive. The table below lists the standard picks.

Worked example. $N = 3, W = 2, R = 2$: $2 + 2 > 3$, so every read touches a replica the write touched and sees the newest write. And $2 \leq 3 - 1$, so reads and writes survive one dead replica. Drop to $W = 1, R = 1$ and a write on replica A is invisible to a read of B and C.

N	W	R	Survives	Reads see writes?	Use when
3	2	2	1 failure	Yes	Default for config, metadata, model registry versions
3	1	3	1 failure for writes	Yes	Write-heavy, reads can afford to ask everyone
3	3	1	1 failure for reads	Yes	Read-heavy, writes can afford to wait
3	1	1	2 failures	No	Caches and scratch data where staleness is fine
5	3	3	2 failures	Yes	Control planes that must survive a rack loss

Decision procedure: start from the failure budget (how many replicas may die), then pick the smallest W and R that satisfy both the overlap rule and the survival rule. Smaller quorums are faster; the rules tell you how small you may go.

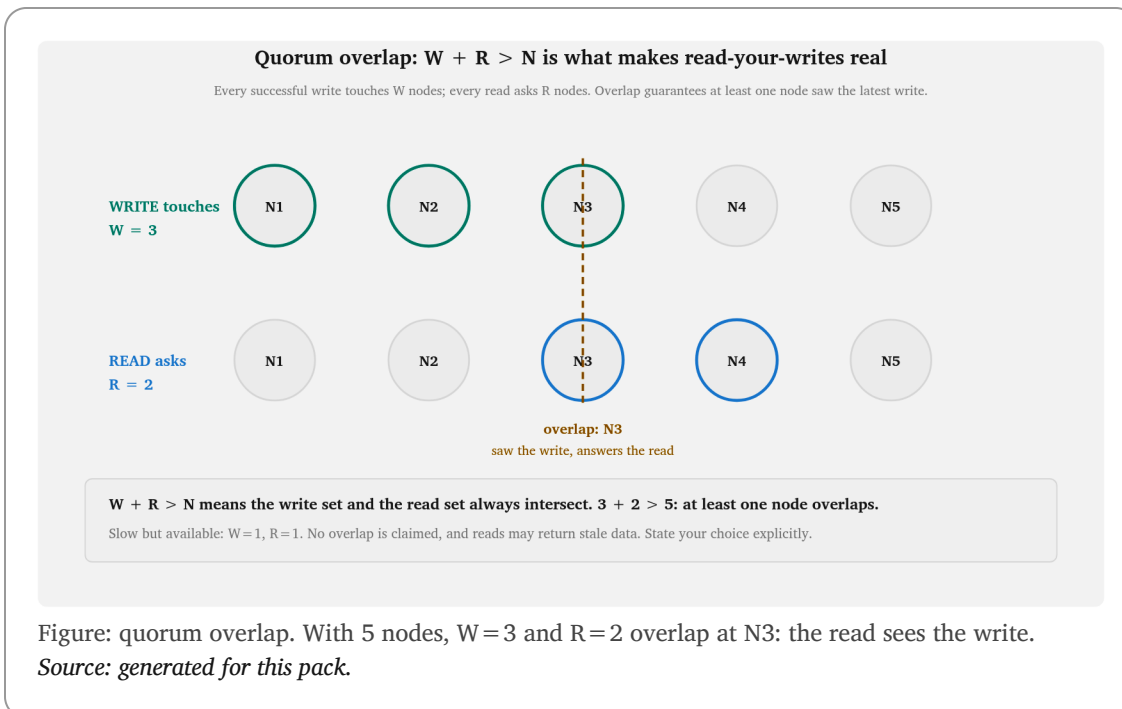
```
# WHAT: the quorum rule as a check you can call in a design review.
# WHY:  $W + R > N$  guarantees every read overlaps every write, so a
#       read sees the latest write. This is the math behind the rule.
# WHAT BREAKS: picking W and R for speed ( $W=1$ ,  $R=1$ ) and silently
#       giving up read-your-write. Fast reads that lie are a bug.

def quorum_ok(n, w, r):
    # WHAT: True when any read overlaps any write (linearizable reads).
    return w + r > n

def survives(n, w, r, failures):
    # WHAT: True when reads and writes still find enough live replicas.
    return w <= n - failures and r <= n - failures

# Balanced 3-copy store: consistent, survives 1 failure.
print(quorum_ok(3, 2, 2), survives(3, 2, 2, 1))    # -> True True
# Write-fast 3-copy store: consistent reads die after 1 failure.
print(quorum_ok(3, 1, 3), survives(3, 1, 3, 1))    # -> True False
# Fast both ways: no overlap, reads can return stale data.
print(quorum_ok(3, 1, 1), survives(3, 1, 1, 1))    # -> False True
# N=5 tolerating 2 failures with strong reads.
print(quorum_ok(5, 3, 3), survives(5, 3, 3, 2))    # -> True True
```

Quorums decide how many replicas you wait for. The next decision is what the waiting buys you: the consistency model, from linearizable down to eventual, picked as weak as the product allows.



How to read this diagram

1. Five nodes, $W=3$, $R=2$. Every write touches three nodes; every read asks two.
2. This pair overlaps at N3. But $W=3$, $R=2$ with $N=5$ does not guarantee it: $3 + 2 = 5$ is not > 5 , so this overlap is luck. For the guarantee, raise R to 3: $3 + 3 = 6 > 5$, and every read quorum overlaps every write quorum.
3. N3 saw the write, so it answers the read. That overlap is the whole guarantee.

1.2 Consistency model selection**Step 1: the tradeoff.**

Not every dataset needs the strongest guarantee. Stronger consistency costs latency and availability, so pick the weakest model that keeps the product correct. The test: ask what breaks if a read is 5 seconds stale. If the answer is "money moves wrong" or "two leaders act at once," you need linearizable. If the answer is "a user sees a slightly old feed," eventual is fine and much cheaper.

Step 2: the ladder.

```
linearizable (consensus: a group of machines agreeing on one value despite
failures; single key)      slowest, strongest
    |
sequential
    |
causal
    |
tunable quorum ( $W + R > N$ )
    |
eventual                      fastest, weakest
```

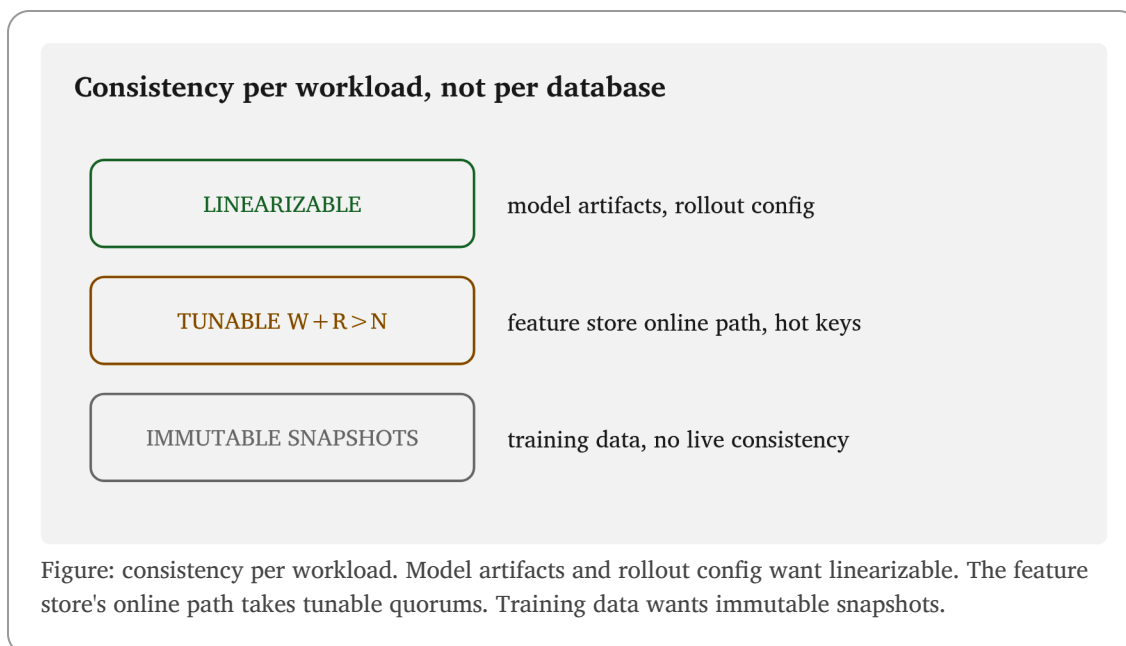
Model artifacts and rollout config want linearizable.

The feature store's online path (a feature store: the shared system that serves features to both training and serving) usually takes tunable quorums.

Step 3: the production picks.

Model	Guarantee	Price	Use for
Linearizable	Reads see the latest completed write, with a real-time order	Quorum on every op, slowest	Leader election, config, model registry versions, billing counters
Sequential / causal	Everyone sees updates in the same order (or cause before effect)	Lighter than linearizable	Chat threads, activity feeds, agent action logs
Eventual	Replicas converge if writes stop; reads may be stale	Cheapest, most available	Training feature snapshots, embedding caches, analytics
Read-committed snapshot	Reads see a consistent point in time	One coordinator	Offline eval datasets, billing reports

For the AI platform in particular, model artifact versions and rollout config want linearizable reads and writes



. The feature store's online path usually takes tunable quorums ($W + R > N$ on hot keys). Training data wants immutable snapshots, not live consistency at all.

The consistency ladder: from strongest to weakest

Stronger costs more latency. Pick the cheapest rung that satisfies the domain rule.

linearizable

behaves like one machine

the strictest, most expensive

sequential

operations look globally ordered

still strong, still costly

causal

causally related writes stay ordered

cheap enough for most feeds

read-your-writes

your own writes are immediately visible

session-scoped, often free

eventual

converges if writes stop

cheapest, weakest

Rule of thumb: money and locks get linearizability; feeds and counters get causal or eventual. Never pay for a rung the product does not need.

Figure: the consistency ladder, from strongest to weakest: linearizable, sequential, causal, read-your-writes, eventual. *Source: generated for this pack.*

How to read this diagram

1. Strongest at top: linearizable behaves like one machine, at the highest cost.
2. Each rung down trades guarantee for cost: sequential, causal, read-your-writes, eventual.
3. Name the guarantee honestly: exactly-once delivery does not exist, only exactly-once effects.

1.3 Queue vs stream**Step 1: the queue: work distribution.**

A queue delivers each message to one consumer and deletes it on acknowledgment. Simple work distribution: each task runs once and nobody replays. Queues are SQS, Celery, Cloud Tasks.

Step 2: the stream: replay and fan-out.

A stream (partitioned log) appends messages durably. Each consumer tracks its own offset and can replay history. Need replay, multiple independent consumers, or per-key ordering? Use a stream: Kafka, Kinesis, Pulsar. Fanning one event to 6 consumers takes one topic on a stream; a queue needs 6 queues or a pub/sub layer you now operate.

QUEUE (each message -> one worker)

```
producer -> [msg1][msg2][msg3] -> worker A gets msg1
                                   worker B gets msg2
ack deletes the message. No replay.
```

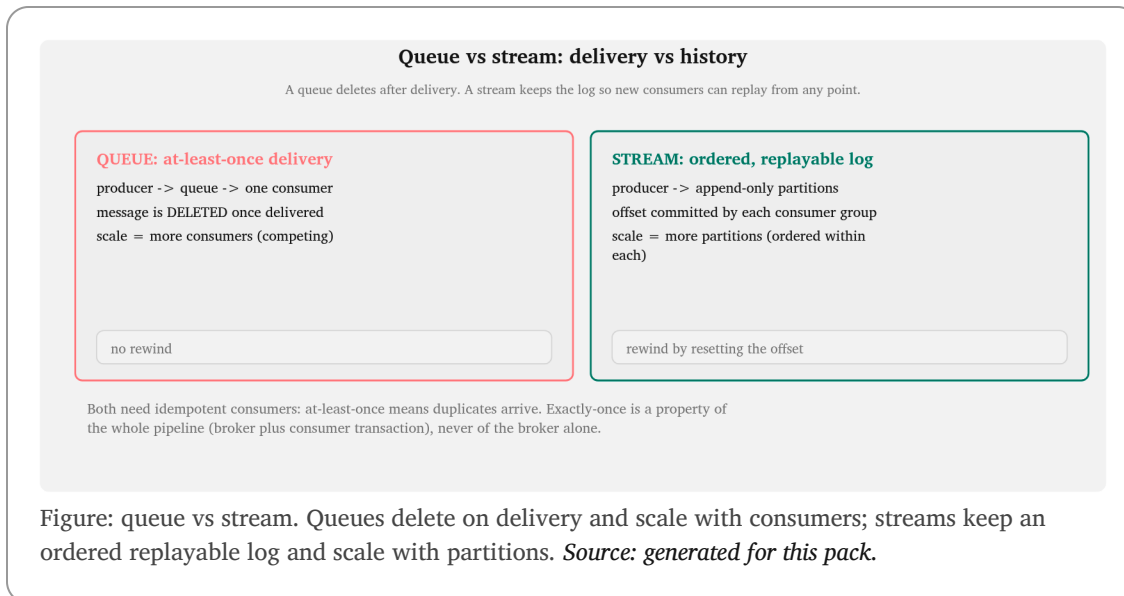
STREAM (append-only log, per-consumer offsets)

```
producer -> log: [e1][e2][e3][e4] ...
                ^         ^
            consumer X   consumer Y
            (offset 2)   (offset 4)
Each consumer moves its own offset. Rewind = replay.
```

Step 3: the price of the stream.

Streams cost more to operate (partitions, retention, consumer-group rebalances (the stream re-assigning partitions when a consumer joins or dies)) and pay back when more than one team reads the same events. A training-data pipeline and a live monitoring consumer reading the same request log is the classic stream-shaped workload.

Queues and streams decide how work moves between stages. The next decision is how fresh the data must be when it arrives: batch, micro-batch, or streaming, picked by the freshness contract. Do not pay streaming prices for batch freshness.



How to read this diagram

1. Queue: the consumer deletes the message once delivered. Scale by adding consumers; there is no rewind.
2. Stream: an ordered, append-only log with per-consumer-group offsets. Scale by adding partitions.
3. Need replay and history: stream. Need at-least-once delivery to one worker: queue.

1.4 Batch vs micro-batch**Step 1: batch: yesterday is fine.**

Ask one question: how stale may the output be? "Yesterday's data is fine" means batch: nightly jobs over the full dataset. Cheapest per event, simplest to reason about, easiest to re-run. A nightly embedding job over 10M documents is batch.

Step 2: streaming: the last few seconds.

Streaming is per-event processing. Most expensive per event and hardest to get right: late data, out-of-order events. A ranking feature that must reflect the last 5 minutes of user behavior needs streaming or tight micro-batch.

Step 3: micro-batch: everything between, and the rule.

Micro-batch means streaming cut into small batches of seconds to minutes. It reuses batch logic with tolerable latency, which is why most "streaming" ML pipelines are really micro-batch. Cost per event rises from batch to micro-batch to streaming; complexity rises faster than cost.

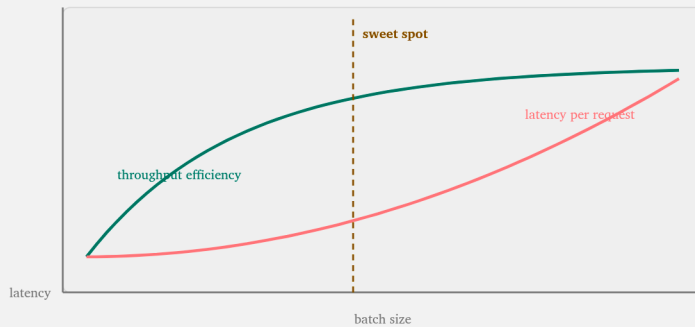
```
batch:      nightly job, 10M docs, hours of freshness, cheapest
micro-batch: minutes of freshness, small windows, more coordination
streaming:  seconds of freshness, per event, most expensive to build and run
```

The freshness contract picks the tier.

Do not pay streaming prices for a batch contract.

Batch vs micro-batch: the latency / efficiency trade

Small batches wait less, move less per trip. Large batches move more, but each request waits longer.



Practice: dynamic batching with a max wait. Batch fills to K or T ms elapses, whichever first.

The SLO caps T. Never tune batch size without stating the latency budget it answers to.

Figure: batch vs micro-batch. Bigger batches raise throughput efficiency but also per-request latency: the sweet spot is where the curves cross the SLO. *Source: generated for this pack.*

How to read this diagram

1. Bigger batches raise throughput efficiency: the GPU stays fed.
2. They also raise per-request latency: requests wait for the batch to fill.
3. The sweet spot is the largest batch whose latency still clears the SLO.

1.5 Cache sizing with Little's Law**Step 1: how many entries: the rule.**

A cache has two sizing questions: how many entries, and how long each lives. Size the hot path with Little's Law (in one line: average concurrency = arrival rate x time in the system; Section 2.1 works it fully): capacity = service rate x maximum acceptable wait. The same arithmetic sizes connection pools, worker counts, and GPU batch slots.

Step 2: the bound, and what happens past it.

A downstream serving 100 requests per second with a 2-second deadline gets a queue bound of 200. Reject everything past it fast (429 or 503) instead of queuing it slow. A request that would wait past its deadline wastes its time and your memory. Little's Law assumes steady state. Bursts need burst math (Section 3.5), not the average.

```

# WHAT: Little's Law, L = arrival_rate * time_in_system.
# WHY:  it turns two measurable numbers (arrival rate, service time)
#       into the one number you must provision (concurrent slots).
# WHAT BREAKS: it assumes steady state. Bursts need burst math
#       (see 3.5, silent queue backlogs), not the average.

def little_slots(arrival_per_s, service_time_s):
    # WHAT: average concurrent jobs in the system. Units: jobs.
    return arrival_per_s * service_time_s

def queue_capacity(service_per_s, max_wait_s):
    # WHAT: bound for a queue with a latency deadline. Units: jobs.
    # WHY:  capacity = rate * wait; past this, reject fast (429/503)
    #       instead of queueing slow.
    return service_per_s * max_wait_s

# 120 req/s, each holding a GPU slot 1.5 s (prefill: processing the prompt
# tokens; decode: generating output tokens one at a time).
print("GPU slots:", little_slots(120, 1.5))          # -> 180.0
# Bounded queue: 100 req/s service, 2 s deadline.
print("queue bound:", queue_capacity(100, 2))        # -> 200

```

Step 3: how long entries live: the TTL is a freshness contract.

If the product tolerates 5-minute-old embeddings, the TTL is 5 minutes. Two refinements. **Jitter the TTLs.** A fixed 5-minute TTL on every key expires everything at once and stampedes the backend (Section 3.3). Spread expiry over 4 to 6 minutes instead. **Serve stale-while-revalidate** for hot keys: return the old value instantly and refresh in the background.

Step 4: how big in bytes: find the knee.

Measure the hit-ratio curve: plot hit ratio against cache size and find the knee where more memory stops buying hits. Size slightly past the knee. Size for the peak working set, not the average: caches fail at peak.

Caches and pools absorb spikes so you do not over-provision for them. The next decision is how many GPUs the serving fleet needs: a one-line formula with three measured inputs.

1.6 GPU fleet sizing

Step 1: three inputs, all measured.

The fleet formula, in one line:

```
GPUs = ceil( Q x (T_in + T_out) / (S x rho) ) + spares
```

Q is requests per second (demand). $T_{in} + T_{out}$ is average prompt plus completion tokens per request. S is tokens per second one GPU sustains at the target batch size. Measure this on your fleet, because a brochure number sizes a fleet that falls over on Monday. ρ is target utilization, 0.7. Spares: at least one per failure domain, so a rack loss does not take the SLO (service-level objective: the product's promised latency/availability line) with it.

Step 2: worked, and the price of 0.9.

40 req/s, 2k in / 500 out, 900 tok/s per GPU measured, util 0.7: 160 GPUs. The same demand at util 0.9 needs 125 GPUs. But the M/M/1 wait (the classic single-server queue model with random arrivals and service times; Section 2.2 derives the curve) at 0.9 is 9x service time versus 2.3x at 0.7. The extra 35 GPUs buy the latency SLO. That tradeoff is the capacity conversation.

```
# WHAT: GPU fleet sizing from demand, supply, and a policy.
# WHY: capacity reviews need one number with named inputs; this is it.
# WHAT BREAKS: S (tokens/s per GPU) must be MEASURED at the target
# batch and context mix. A brochure number sizes a fleet that
# falls over on Monday.

import math

def gpu_fleet(q_per_s, t_in, t_out, tokens_per_s_per_gpu,
              target_util=0.7, spares=1):
    # WHAT: ceil(demand / (supply * util)) + spares. Units: GPUs.
    demand = q_per_s * (t_in + t_out)      # tokens/s the fleet must emit
    usable = tokens_per_s_per_gpu * target_util
    return math.ceil(demand / usable) + spares

# 40 req/s, 2k in / 500 out, 900 tok/s per GPU measured, util 0.7.
print("GPUs:", gpu_fleet(40, 2000, 500, 900))    # -> 160
# Same demand at util 0.9 needs 125 GPUs, but the M/M/1 wait at
# 0.9 is 9x service time versus 2.3x at 0.7. The extra 35 GPUs
# buy the latency SLO. That tradeoff is the capacity conversation.
print("GPUs at 0.9:", gpu_fleet(40, 2000, 500, 900, target_util=0.9))
```

The price of 0.9: GPUs vs the wait

Same demand: 40 req/s, 2k in / 500 out, 900 tok/s per GPU measured. The extra 35 GPUs buy the latency SLO.

util 0.7  160 GPUs, wait 2.3x service

util 0.9  125 GPUs, wait 9x service

Saving 35 GPUs (about 22%) costs a 4x latency blowup. The M/M/1 hockey stick is why fleets target 60-75%.

Rule: never size a fleet to 100% of measured supply.

Figure: the price of 0.9. Squeezing the same demand into 125 GPUs instead of 160 saves about 22% of the fleet but multiplies the queue wait by 4x. The M/M/1 hockey stick is why fleets target 60-75% utilization. *Source: generated for this pack.*

```
fleet = ceil(demand / (supply x 0.7)) + spares
```

```
120 req/s demand / (40 req/s per GPU x 0.7) = 4.3 → 5 GPUs
```

```
5 + 1 spare = 6 GPUs
```

0.7 is the headroom factor. Never size to 100% of measured supply.

Key takeaways

- Quorums: $W + R > N$ for correct reads; $W, R \leq N - f$ to survive f failures.
- Consistency: weakest model that keeps the product correct; money and leadership need linearizable.
- Queue for work distribution, stream for replay and fan-out.
- Freshness contract picks batch, micro-batch, or streaming. Do not pay streaming prices for batch freshness.
- Little's Law sizes queues, pools, and GPU slots: $\text{capacity} = \text{rate} \times \text{wait}$.
- GPU fleet = $\text{ceil}(\text{demand} / (\text{supply} \times 0.7)) + \text{spares}$, with measured supply.

Next: Part 2 gives you the formulas worth memorizing: Little's Law, queue waits, KV-cache math, burn rates, and idempotency.

Part 2. Must-memorize numbers and formulas

Part 1 gave you the rules. This part gives you the formulas worth carrying in your head. Each entry gives the formula, its units, what it means in plain terms, and a worked number.

2.1 Little's Law: $L = \lambda W$

Step 1: the intuition.

The average number of jobs inside a system equals the arrival rate times the average time each job spends inside. A checkout line with 2 shoppers arriving per minute, each taking 3 minutes, holds 6 shoppers on average. It holds for any steady-state system: queues, thread pools, GPU batch slots.

Step 2: the formula, the units, the worked numbers.

$L = \lambda \times W$. Units: jobs = (jobs/second) x seconds.

$$L = \lambda \times W$$

where: **L** means the average number of jobs inside the system. **λ (lambda)** means the arrival rate in jobs/second. **W** means the average time each job spends inside, in seconds. Holds for any steady-state system: queues, thread pools, GPU batch slots. The code below works it three ways. $120 \times 1.5 = 180$ GPU slots. A $100 \text{ req/s} \times 2 \text{ s}$ queue bound of 200. A cache in-flight allowance.

```
# WHAT: Little's Law worked three ways: L = arrival_rate x time_in_system.
Units: jobs.
# WHY: two measurable numbers (arrival rate, service time) become the one
#       number you provision (concurrent slots).
# WHAT BREAKS: steady state only. Bursts need burst math (Section 3.5).

print("GPU slots:", 120 * 1.5)           # -> 180.0
print("queue bound:", 100 * 2)          # -> 200
print("inflight allowance:", 40000 * 0.25) # -> 10000.0 (cache twin)
```

Read them: 120 requests/s x 1.5 s per request = 180 concurrent slots. With 8 slots per GPU (batch 8: requests processed together in one GPU pass), that is 23 GPUs before headroom. A 100 req/s service with a 2 s deadline gets a queue bound of 200. A cache twin at 40k reads/s through a 250 ms refresh window allows 10,000 in flight.

Step 3: see it on a cache.

Little's Law: how many slots the cache must hold

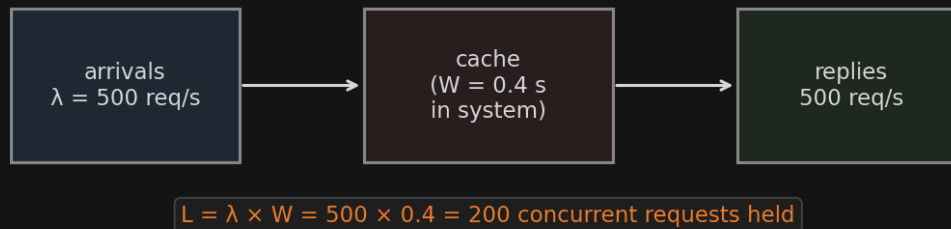
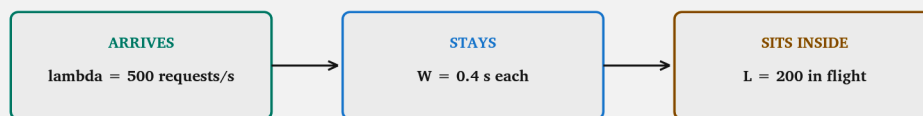


Figure: Little's Law on a cache: 500 req/s arriving and 0.4 s in the system means 200 concurrent slots must exist. Size pools, queues, and GPU slots with $L = \lambda W$. *Source: generated for this pack.*

Boundary: steady state only. During a burst, arrivals exceed service and the queue grows without bound. Size for the burst (Section 3.5), not the average.

Little's Law: $L = \text{lambda} \times W$, read as a flow

What sits in the system equals what arrives per second times how long each request stays.



$L = 500 \times 0.4 = 200$. Capacity planning reads right to left: 200 slots needed to hold 0.4 s of work at 500/s.

If queues grow past L , either arrival rate rose or service time did. Measure which before scaling anything.

Assumes a stable system: arrival rate roughly equals completion rate over the window.

Figure: Little's Law as a flow. 500 requests/s arriving, 0.4 s each inside: 200 in flight. *Source: generated for this pack.*

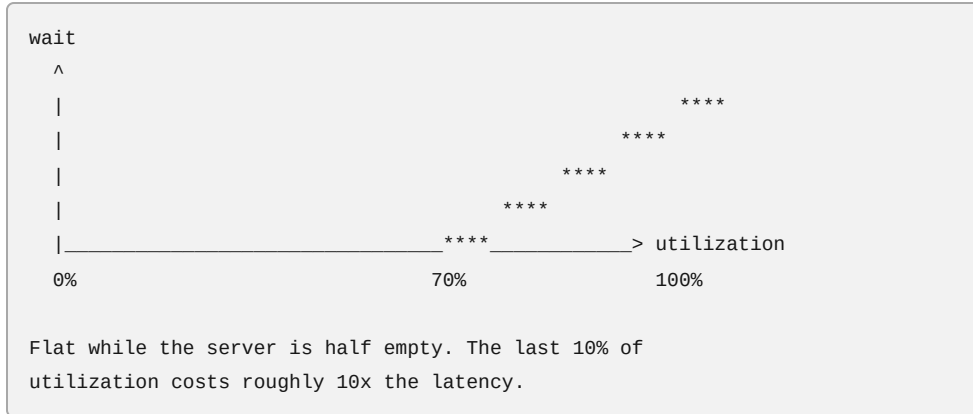
How to read this diagram

1. Lambda: 500 requests arrive per second. W: each stays 0.4 seconds.
2. L: 200 are inside the system at once. That is the cache size the math demands.
3. Little's Law does accounting, not theory: if you know any two, the third follows.

2.2 M/M/1 wait times: $\rho/(\mu(1-\rho))$

Step 1: the shape of the wait.

The M/M/1 model is the classic picture of one server fed by random arrivals with random service times (the M's stand for memoryless, the 1 for one server). With random arrivals and random service times, the wait before service starts stays flat while the server is half empty. Then it bends up like a hockey stick. The last 10 percent of utilization costs roughly 10x the latency.



Step 2: the formula and the numbers.

Mean queue wait = $\rho / (\mu \times (1 - \rho))$, in seconds. ρ is utilization (λ/μ , a number from 0 to 1). μ is the service rate in jobs/second. Worked at 100 ms mean service time:

$$\text{mean queue wait} = \rho / (\mu \times (1 - \rho))$$

where: ρ (**rho**) is utilization (λ/μ , a number from 0 to 1); μ (**mu**) is the service rate in jobs/second. The table below works it at 100 ms mean service time. The wait stays flat while the server is half empty, then bends up like a hockey stick.

Utilization ρ	Mean queue wait	As multiple of service
0.50	100 ms	1x
0.80	400 ms	4x
0.90	900 ms	9x
0.95	1900 ms	19x

```
# WHAT: M/M/1 mean queue wait: rho / (mu * (1 - rho)), in seconds.
# WHY:  this one curve explains why fleets target ~70% utilization:
#       the wait explodes as rho approaches 1. Textbook result.
# WHAT BREAKS: real arrivals are burstier than Poisson, so this is
#             a LOWER bound. Budget above it, never below.

def mm1_wait_s(rho, service_time_s):
    # WHAT: mean time a job waits before service starts. Units: seconds.
    mu = 1.0 / service_time_s
    return rho / (mu * (1.0 - rho))

svc = 0.100 # 100 ms mean service time
for rho in (0.5, 0.8, 0.9, 0.95):
    w = mm1_wait_s(rho, svc)
    print(f"rho={rho:.2f}: mean queue wait {w*1000:6.1f} ms "
          f"= {w/svc:4.1f}x service time")

# Verified output:
# rho=0.50: mean queue wait 100.0 ms = 1.0x service time
# rho=0.80: mean queue wait 400.0 ms = 4.0x service time
# rho=0.90: mean queue wait 900.0 ms = 9.0x service time
# rho=0.95: mean queue wait 1900.0 ms = 19.0x service time
```

Step 3: the production rule.

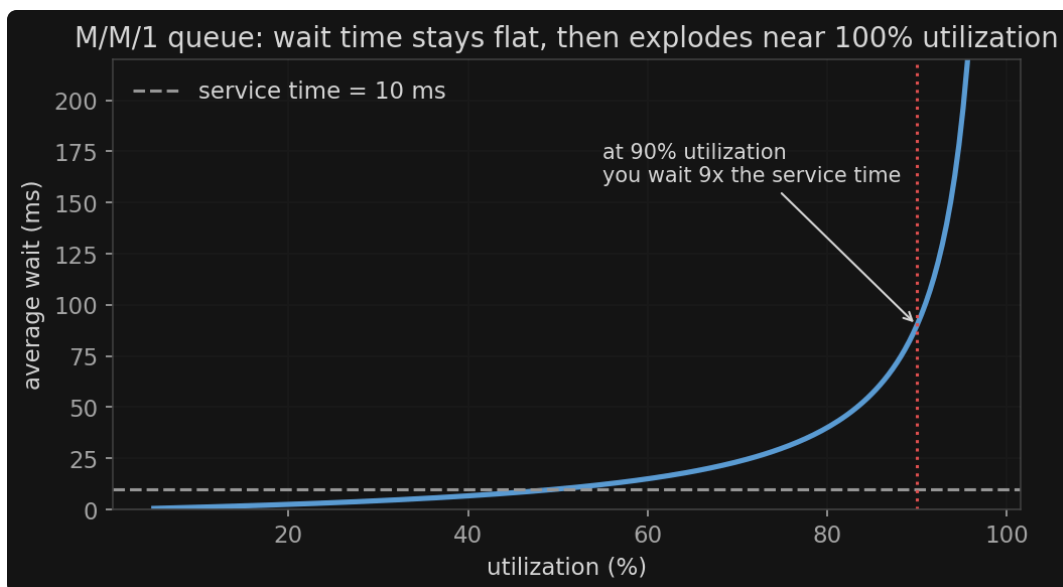


Figure: average queue wait vs utilization for a 10 ms service time. Flat until 70%, then the hockey stick: at 90% you wait 9x the service time. Never size a fleet to 100% of measured supply. *Source: generated for this pack.*

Target 60 to 75 percent utilization in production; treat 85 percent as an incident. Real traffic is burstier than the Poisson (purely random, memoryless) arrivals this model assumes, so treat these waits as optimistic lower bounds: budget above them, never below.

Queues tell you how long work waits. The next section is about finding things fast: HNSW recall parameters, and what they cost in memory and latency.

2.3 HNSW recall parameters

Step 1: start with brute force.

Below about 100k vectors, brute force in memory is usually simpler and has perfect recall: compare the query against every vector. Past that, $O(N)$ per query (cost growing in direct proportion to the number of vectors) gets expensive, and you want an index.

Step 2: the layered graph.

HNSW (hierarchical navigable small world) is the standard approximate nearest-neighbor index for vector search. Vectors sit in layered graphs. Cheap greedy hops (greedy: always step to the nearest-looking neighbor) on the sparse upper layers find the right neighborhood. Then a wide beam search (a shortlist of the best candidates, expanded from all of them at once) on the dense bottom layer collects candidates. Search turns from $O(N)$ into roughly $O(\log N)$ per query.

HNSW: a hierarchy of proximity graphs; efSearch widens the beam at the bottom layer

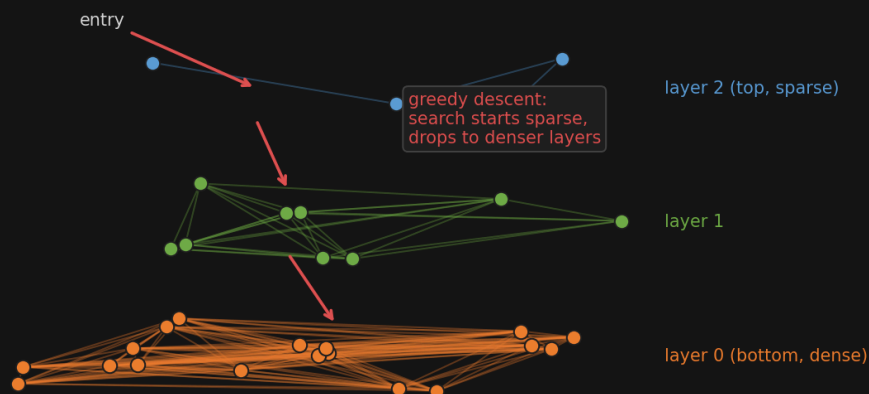


Figure: HNSW descends from the sparse top layer to the dense bottom layer by greedy hops. efSearch widens the candidate beam at the bottom layer. That trades recall against latency. *Source: generated for this pack.*

Step 3: the three knobs, and the tuning loop.

- **M** (typical 16): max edges per node per layer. Sets graph connectivity and memory. Higher M helps recall at high dimensions and costs memory.
- **ef_construction** (typical 200): beam width at build time. Higher is slower to build, better graph quality.

- **ef / ef_search** (typical 50 to 200): beam width at query time. This is the live recall knob.

recall@k is the fraction of the true k nearest neighbors the index returns. The tuning loop: measure recall@k against brute force on a sample of real queries. Start ef at 50. Double until recall@k reaches 0.95 or the latency budget runs out. If you run out of budget first, shard the index or quantize it, not a bigger ef.

Memory: vectors cost $\text{dim} \times 4$ bytes each (FP32); the graph costs about $M \times 2 \text{ edges} \times 4$ bytes per node. Worked: 1M vectors, dim 768, $M = 16$. The math: $1\text{M} \times 768 \times 4 = 3.07$ GB for vectors plus $1\text{M} \times 16 \times 2 \times 4 = 128$ MB for the graph. The vectors dominate; graph overhead is under 5 percent.

2.4 KV-cache math: 2 x layers x hidden x bytes

Step 1: why the cache exists.

During generation, each new token attends to every previous token (attention: the mechanism by which each generated token weighs the previous tokens). Recomputing all of them at every step is pure waste, so the model caches the keys and values of every previous token. The cache lives in GPU memory next to the weights and grows with context length and batch size.

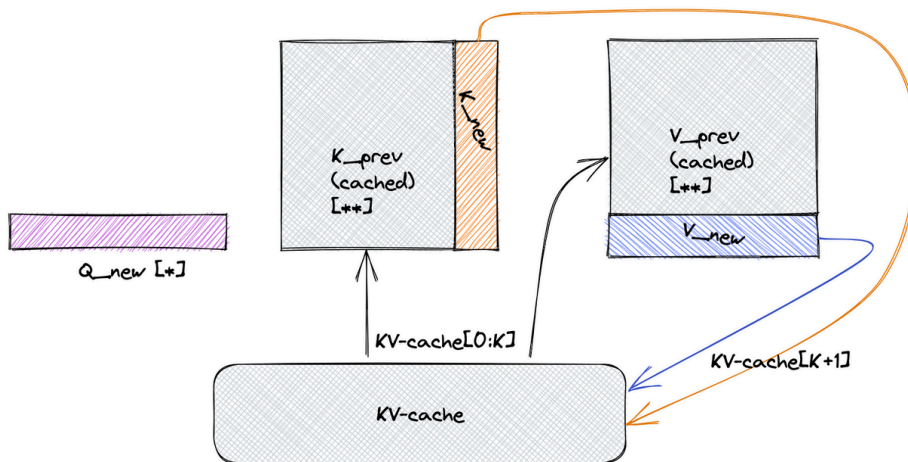


Figure: what lives in the KV cache. Each decoding step attends the new query to cached keys and values, then appends the new key and value so the past is never recomputed. *Source: existing curriculum diagram.*

Step 2: the formula.

Bytes per token, in one line:

$$\text{bytes per token} = 2 (K \text{ and } V) \times \text{layers} \times \text{hidden_dim} \times \text{bytes_per_value}$$

For an 8B model (32 layers, hidden 4096, FP16): $2 \times 32 \times 4096 \times 2 = 524,288$ bytes, about 0.5 MB per token. An 8k context needs about 4.3 GB of cache.

Step 3: the batch picture, where fleets die.

A batch of 32 such requests needs about 137 GB of cache. That is more than an 80 GB GPU holds, before counting the 16 GB of weights. Long contexts, not model size, are usually what runs a serving fleet out of memory. PagedAttention (vLLM) fights the fragmentation: it splits each sequence's cache into fixed-size blocks, so waste shrinks to the last block only.

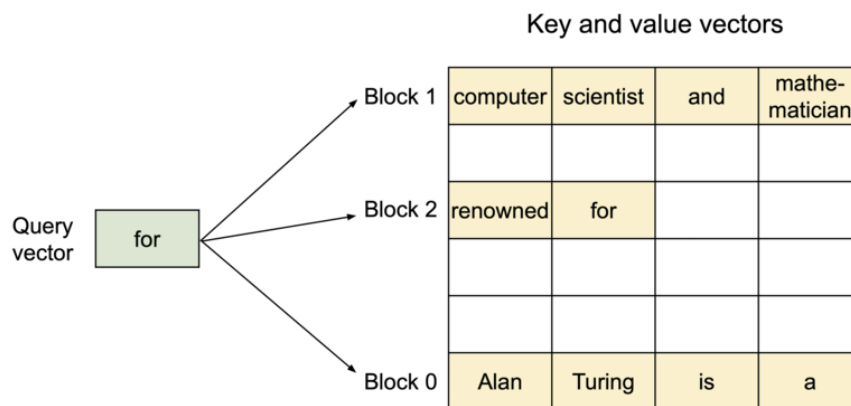


Figure: PagedAttention splits each sequence's KV cache into fixed-size blocks. The query for one token reads blocks 1, 2, and 0, which need not sit contiguously in memory. Waste shrinks to the last block only. *Source: vLLM project, UC Berkeley (vllm.ai/blog/vllm).*

Step 4: check the model card.

The formula assumes standard multi-head attention. Grouped-query attention stores fewer key/value heads and shrinks the cache; check the model card before sizing.

Attention Variants

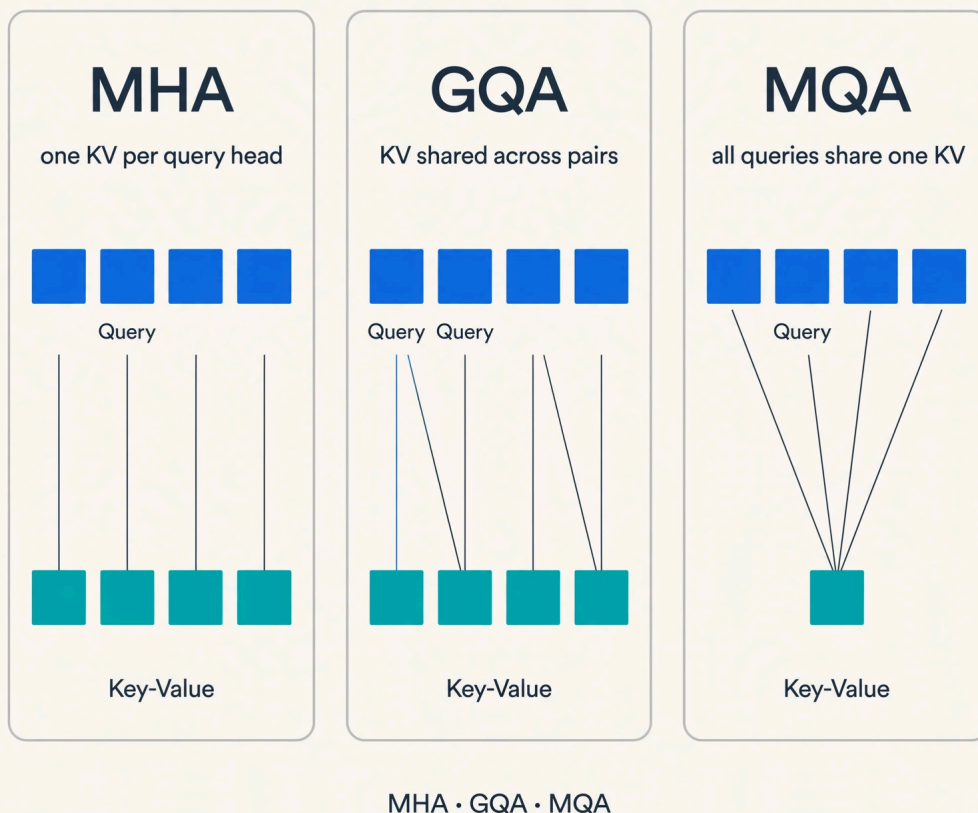


Figure: grouped-query attention shares key/value heads across query heads, shrinking the KV cache that this section's formula sizes. Check the model card: the $2 \times \text{layers} \times \text{hidden} \times \text{bytes}$ rule assumes standard multi-head attention. *Source: existing curriculum diagram.*

2.5 The 2-FLOP-per-token rule

Step 1: the rule.

A forward pass costs about 2 floating-point operations per parameter per token: one multiply and one add per weight. For an 8B model that is 16 GFLOP per token.

Step 2: the compute floor.

16 GFLOP (billions of floating-point operations) / 300 TFLOPs (trillions per second) effective = about 53 microseconds per token. But real decoding is memory-bound: each step streams all the weights from GPU memory, and bandwidth (not FLOPs) is the binding constraint. Treat the FLOP math as a floor, not a promise. Prompt processing (prefill) is closer to compute-bound, so the rule gives usable throughput estimates there. Batching helps for the same reason: one weight-streaming pass serves many tokens at once.

Step 3: the bit ladder.

```
8B params x 2 FLOP/param/token = 16 GFLOP per token
```

```
FP16 weights: 8B x 2 bytes = 16 GB
```

```
INT4 weights: 8B x 0.5 bytes = 4 GB
```

```
Halve the bits, halve the memory.
```

Halving the bytes per parameter (FP16 to INT8 to INT4) halves weight memory and roughly doubles memory-bound throughput, at the cost of accuracy cliffs you must version and test.

```
# WHAT: the ML-literacy formulas a systems engineer reaches for.
# WHY: capacity reviews for serving fleets reduce to these three:
#     memory per token, compute per token, and the weights.
# WHAT BREAKS: the KV-cache line assumes standard multi-head
#     attention. Grouped-query attention stores fewer KV heads,
#     which shrinks the cache; check the model card.

def kv_cache_bytes_per_token(layers, hidden, bytes_per_value=2.0):
    # WHAT: 2 (K,V) x layers x hidden x bytes. Units: bytes/token.
    return 2 * layers * hidden * bytes_per_value

def forward_flops_per_token(params):
    # WHAT: ~2 FLOP per parameter per token (one multiply-add).
    return 2 * params

def model_bytes(params, bytes_per_param=2.0):
    # WHAT: weight memory. FP16=2.0, INT8=1.0, INT4=0.5 bytes/param.
    return params * bytes_per_param

# 8B model, 32 layers, 4096 hidden, FP16.
bpt = kv_cache_bytes_per_token(32, 4096)
print(f"KV cache: {bpt/1e6:.2f} MB/token")          # -> 0.52 MB/token
print(f"8k context: {bpt*8192/1e9:.1f} GB")         # -> 4.3 GB
print(f"weights FP16: {model_bytes(8e9)/1e9:.0f} GB") # -> 16 GB
print(f"weights INT4: {model_bytes(8e9, 0.5)/1e9:.1f} GB") # -> 4.0 GB
print(f"forward: {forward_flops_per_token(8e9)/1e9:.0f} GFLOP/token") # -> 16
```

The FLOP and memory math sizes the fleet. The next section watches it burn: burn-rate alerting on the error budget, paging on pace, not on raw counts.

The 2-FLOP-per-token rule

A dense transformer forward pass costs about 2 multiply-adds per parameter per token. Enough to size any serving job on a napkin.

7B model, 1 token

$2 \times 7e9 \times 1$

14 GFLOP

7B model, 1k tokens

$2 \times 7e9 \times 1000$

14 TFLOP

70B model, 1k tokens

$2 \times 70e9 \times 1000$

140 TFLOP

Divide by the GPU's usable FLOPs to get a floor on time: 140 TFLOP on an 800-TFLOP card is at least ~0.2 s of pure compute.

Attention adds $O(n^2)$ memory traffic on top; for long contexts the KV cache, not the FLOPs, is usually the binding constraint.

Dense-model rule only. MoE activates a fraction of parameters per token, so use active parameters, not total.

Figure: the 2-FLOP-per-token rule. A 7B model burns 14 GFLOP per token, 14 TFLOP per 1k tokens; 70B burns 140 TFLOP per 1k. *Source: generated for this pack.*

How to read this diagram

1. The rule: about 2 FLOP per parameter per token.
2. 7B: 14 GFLOP per token, 14 TFLOP per 1k tokens.
3. 70B: 140 TFLOP per 1k tokens. Size the fleet from this number before anything else.

2.6 Burn-rate alerting**Step 1: why raw thresholds fail.**

"Alert me when latency exceeds 2 seconds" pages you at 3am for one slow request. It misses a real degradation holding 30 percent of requests at 1.9 seconds. Threshold alerts ignore the error budget (the allowed failure slice of the SLO: 0.1% of requests for a 99.9% SLO).

Two burn-rate windows: page now, ticket soon

fast window

pages: budget drains fast

slow window

tickets: slow leak

Threshold alerts ignore the budget. Burn-rate alerts spend it.

Error budget: the allowed failure slice of the SLO.

Figure: burn-rate windows. The fast window pages when the budget drains fast. The slow window files tickets on a slow leak. Threshold alerts ignore the budget.

Step 2: the burn rate.

Burn rate = **observed error ratio** / **budgeted error ratio**. A 99.9 percent SLO allows 0.1 percent errors; errors running at exactly 0.1 percent burn at 1x. At 10x, the monthly budget is gone in 3 days. At 2x, it lasts 15 days.

Step 3: two alerts, two urgencies.

- **Page tier:** 10x burn over 5 minutes. At 10x, the monthly budget is gone in 3 days. That pages.
- **Ticket tier:** 2x burn over 1 hour. At 2x, the budget lasts 15 days. That tickets, no page.
- Two alerts, two urgencies, zero 3am pages for single blips.

```

# WHAT: burn-rate math for SLO alerting.
# WHY:  thresholds page on single blips and miss slow leaks; burn rate
#       ties the alert to the error budget, so urgency is automatic.
# WHAT BREAKS: a burn-rate alert needs a window. Too short (1 min)
#       pages on noise; too long (6 h) pages after the budget is gone.

def burn_rate(observed_error_ratio, slo_error_budget_ratio):
    # WHAT: budget consumption speed. 1.0 = exactly on budget.
    #       Units: dimensionless multiple.
    return observed_error_ratio / slo_error_budget_ratio

def budget_days_left(burn, window_days=30.0):
    # WHAT: days until the budget is gone at this burn.
    return window_days / burn if burn > 0 else float("inf")

SLO = 0.001 # 99.9% SLO allows 0.1% errors
# Page tier: 1% errors sustained 5 min -> burn 10x.
b = burn_rate(0.01, SLO)
print(f"burn {b:.0f}x -> budget gone in {budget_days_left(b):.1f} days")
# -> burn 10x -> budget gone in 3.0 days. Pages.
# Ticket tier: 0.2% errors over an hour -> burn 2x.
b = burn_rate(0.002, SLO)
print(f"burn {b:.0f}x -> budget gone in {budget_days_left(b):.1f} days")
# -> burn 2x -> budget gone in 15.0 days. Tickets, no page.

```

Nines table, worth memorizing (30-day month = 43,200 minutes):

SLO	Allowed downtime/month
99%	7.2 hours
99.9%	43 minutes
99.99%	4.3 minutes
99.999%	26 seconds

Step 4: see the alert windows.

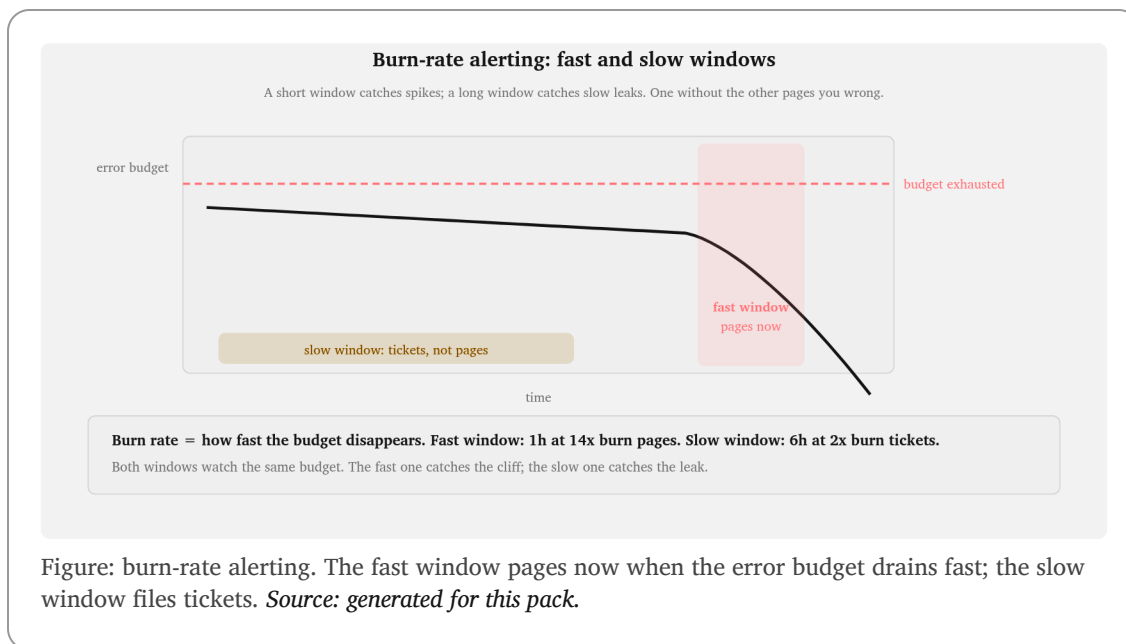
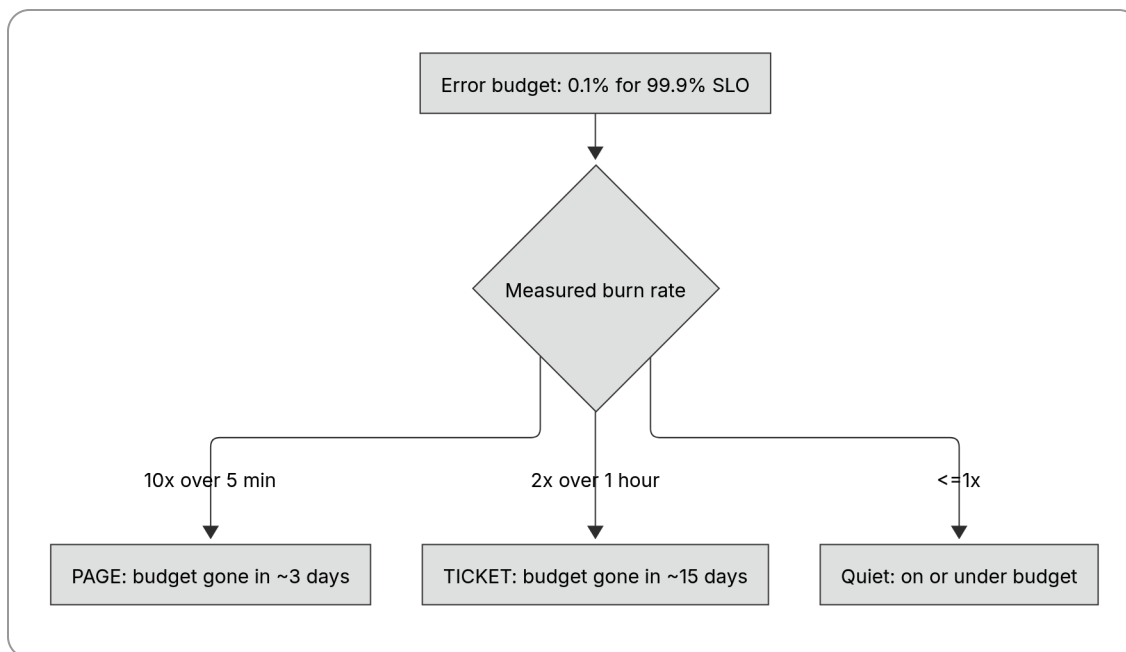


Figure: burn-rate alerting. The fast window pages now when the error budget drains fast; the slow window files tickets. *Source: generated for this pack.*

How to read this diagram

1. The fast window pages now: the error budget drains quickly.
2. The slow window files tickets: a slow leak that still matters.
3. Time runs left to right; the budget line is the promise to the user.

2.7 Idempotency patterns

Step 1: the problem.

Networks fail mid-request, so clients retry, and the server cannot tell a retry from a new request. Charging the customer twice is the failure mode.

Step 2: the key-to-result table.

The fix is an **idempotency key**: a client-generated unique token sent with the request. The server keeps a table of key to result. A repeated key returns the stored result instead of running the job again.

```
client sends key K with the request

server: look up K in key->result table (TTL past the retry window)
  hit  → return the stored result, do nothing again
  miss → execute, store result under K, return it

Retries become safe when the second arrival is a lookup, not a replay.
```

```
# WHAT: idempotency keys, the receiver half of "effectively once".
# WHY:  networks fail mid-request, so clients retry. The server keeps
#       key -> result; a repeated key returns the stored result
#       instead of running the job again.
# WHAT BREAKS: a TTL shorter than the longest retry window lets a
#       late duplicate re-execute. TTL must exceed (attempts x backoff).

class IdempotentStore:
    def __init__(self):
        self._results = {} # key -> result, with a TTL in production

    def run_once(self, key, fn, *args):
        # WHAT: run fn at most once per key; repeats return the stored
        #       result. Units: one result per unique key.
        if key in self._results:
            return self._results[key], "duplicate"
        result = fn(*args)
        self._results[key] = result
        return result, "executed"

calls = []
store = IdempotentStore()
for _ in range(3): # client retries twice after timeouts
    out, how = store.run_once(
        "job-9f31",
        lambda t: calls.append(t) or f"embedding({t})", "text-3")
    print(how, "->", out)
print("side effects:", len(calls))
# Verified output:
# executed -> embedding(text-3)
# duplicate -> embedding(text-3)
# duplicate -> embedding(text-3)
# side effects: 1
```

Step 3: the two details that break it.

Key scope: one key per logical operation, generated by the client before the first attempt, reused across retries. **TTL:** the table entry must outlive the longest retry window (attempts x backoff cap). A TTL shorter than that lets a late duplicate re-execute. This table is the receiver half of the effectively-once story in Section 3.2.

Key takeaways

- Little's Law: $L = \lambda W$, in jobs. Sizes queues, pools, slots.
- M/M/1: $\text{wait} = \rho / (\mu(1-\rho))$. At 0.9, 9x service time. Target 0.6 to 0.75.
- HNSW: $M \sim 16$, $\text{ef_construction} \sim 200$, tune ef_search (50-200) against recall@k .
- KV cache: 2 x layers x hidden x bytes per token. Context length OOMs fleets.
- 2 FLOP per param per token. Quantization halves memory per bit-ladder step.
- Burn rate: 10x over 5 min pages, 2x over 1 h tickets.
- Idempotency: client key + server key $\rightarrow$ result table with TTL beyond the retry window.

Next: Part 3 catalogs the classic traps: five outage shapes with their fixes.

Idempotency keys: make retries safe

The client sends a unique key with the request. The server stores the result under the key, so a retry returns the stored result instead of doing the work twice.



The key must be unique per logical operation and stored atomically with the result, or the race between two concurrent retries charges twice.

Keys expire: keep them long enough to cover the longest plausible retry storm, then let them go.

Figure: idempotency keys make retries safe. The client generates key K; the server charges once and returns the stored result on retry. *Source: generated for this pack.*

How to read this diagram

1. The client generates key K and sends pay(K).
2. The server sees K the first time: charges, stores result[K].
3. The retry sends the same K after a network hiccup: the server returns the stored result instead of charging twice.
4. The warning at the bottom: concurrent retries without the key check can still charge twice.

Part 3. Classic traps

The formulas describe the system working. This part describes it failing. Five traps that have caused real outages. Each one looks reasonable until it fails, which is what makes it a trap.

3.1 Split-brain without fencing

Step 1: the partition.

A network partition splits the cluster. Both sides elect a leader. The partition heals. Now two leaders have both written, and the data is corrupt in ways no quorum can untangle afterward.

Step 2: election is not enough.

The mistake: treating leader election as sufficient. It is not. "We have Raft (the standard consensus protocol for leader election), so we cannot have two leaders" is the trap version. Raft prevents two leaders in the same term (one numbered election round). But a partitioned old leader that never learns about the new term can still issue writes to followers it can reach.

Step 3: fencing tokens.

Every write carries a token from a monotonic sequencer (a tiny strongly-consistent service, or a database row with an atomic increment). The store only accepts a write whose token exceeds every token it has seen. When the new leader takes over, it fetches a fresh, larger token. The store rejects the old leader's writes, which arrive with smaller tokens. Split-brain becomes harmless: the old leader shouts into a void.

Fencing: the old leader shouts into a void

old leader: token 41

new leader: token 42

store accepts only tokens above the highest seen

old leader's writes (41) die on arrival

new leader's writes (42) land

Without fencing, the healed cluster has two divergent histories.

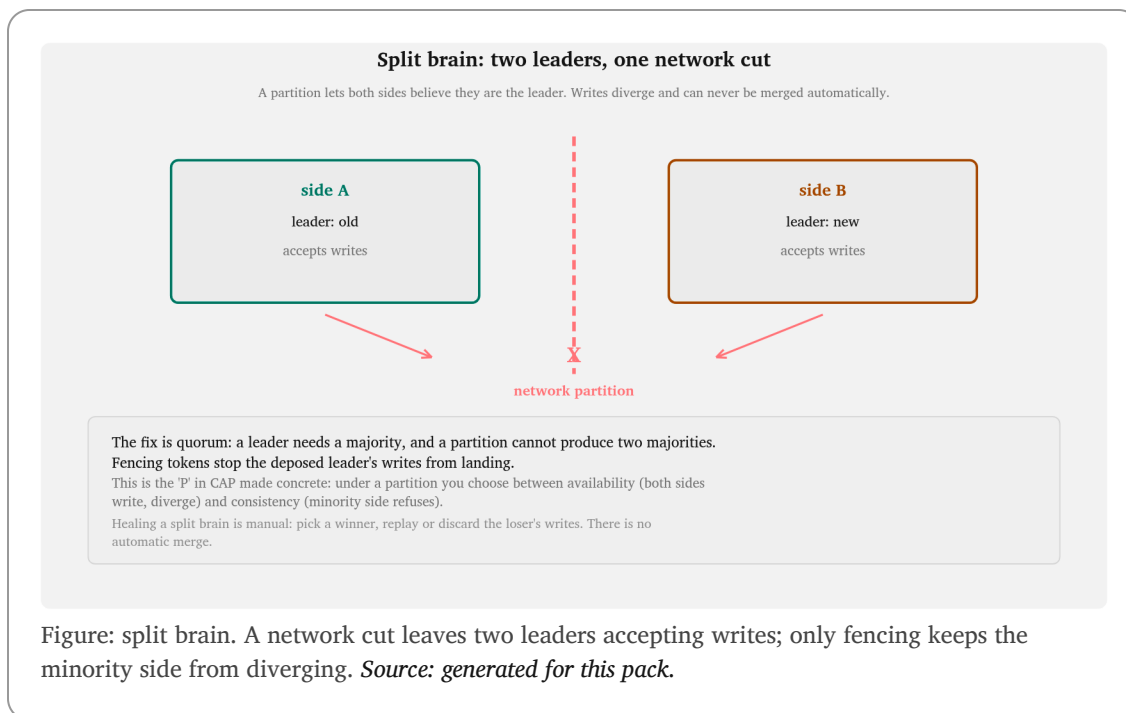
Figure: fencing tokens. The new leader holds token 42; the old leader holds 41. The store accepts only tokens above the highest seen, so the old leader's writes die on arrival.

FENCING TIMELINE (partition, then heal)

```
time -->
leader A (token 41) writes..... | partition
                                | leader B elected, token 42
                                | B writes accepted (42 > 41)
partition heals: A (still token 41) tries to write
  store: 41 <= 42 seen -> REJECTED
Result: one writer won, before the conflict, not after.
```

Step 4: the audit question.

Fencing is the second lock on the door. Any design with a leader needs to answer: **what rejects the old leader's writes?**



How to read this diagram

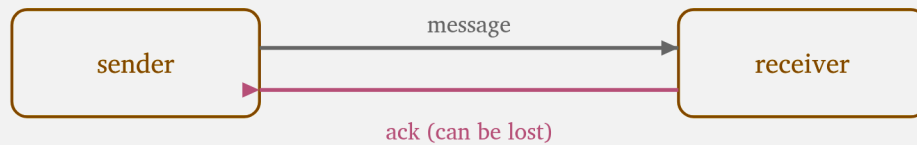
1. A network partition cuts the cluster: side A keeps the old leader, side B elects a new one.
2. Both accept writes. The data diverges.
3. Fencing keeps the minority side from serving writes; without it there is no automatic merge, only a conflict to resolve by hand.

3.2 Exactly-once illusions

Step 1: the impossible.

"No, our pipeline is exactly-once" is the most expensive sentence in data engineering. Over a network that can drop, delay, or duplicate packets, exactly-once delivery is impossible.

Exactly-once delivery is impossible: the Two Generals



The sender can never be sure the receiver got it exactly once.

No protocol fixes this. Name it honestly: **effectively-once**.

Figure: the Two Generals problem. The sender can never be sure the receiver got the message exactly once, because the acknowledgment itself can be lost. No protocol fixes this.

▼ How to read this diagram

1. The sender sends the message to the receiver.
2. The receiver sends an acknowledgment, which the network can lose.
3. The sender can never be sure the receiver got the message exactly once.
4. What it is: the impossibility result behind delivery semantics. Why it exists: networks drop, delay, and duplicate. What breaks: claiming exactly-once instead of naming effectively-once.

The sender can never be sure the receiver got the message exactly once, because the acknowledgment itself can be lost. This is the Two Generals problem (the classic result: two parties cannot guarantee agreement over an unreliable link), and it has no protocol fix.

Step 2: what exists.

- **At-most-once:** fire and forget. Fast, lossy. Metrics and logs can live here.
- **At-least-once:** retry until acknowledged. Nothing is lost, some things run twice. The default for anything important.
- **Effectively-once:** at-least-once delivery plus an idempotent receiver (Section 2.7). The effect happens once even though the delivery happened twice. This is what people mean when they say exactly-once, and naming it honestly keeps the team from forgetting the dedupe logic.

Step 3: what narrows the window, and what closes it.

Kafka's idempotent producer (per-partition sequence numbers) kills duplicates from producer retries. Its transactions make consume-process-produce atomic within the log. Both narrow the window. Neither removes the need for receiver-side idempotency at the bound-

ary where the pipeline touches the outside world (charging, sending email, writing to a third-party API).

"exactly-once" = at-least-once delivery + idempotent receiver
+ dedupe key with a TTL

Delivery semantics: name the guarantee honestly

at-most-once

fire and forget. Fast, lossy. Metrics and logs can live here.

at-least-once

retry until acknowledged. Nothing is lost, some things run twice. The default for anything important.

effectively-once

at-least-once delivery + idempotent receiver. The effect happens once even though delivery ran twice. This is what people mean by exactly-once.

Figure: name the guarantee honestly. Exactly-once delivery does not exist; effectively-once (at-least-once plus an idempotent receiver) is the real target. *Source: generated for this pack.*

The audit question for any "exactly-once" claim: where is the dedupe key, and what is its TTL?

3.3 Cache stampedes

Step 1: the death spiral.

A hot key expires. In the next 100 ms, 5,000 requests miss the cache and hit the database at once. The database falls over, which causes more timeouts, which cause more retries, which deepen the outage. The cache protected the backend; its synchronized expiry removed the protection all at once.

Step 2: the first fix: one backend call, not 5,000.

Request coalescing (singleflight). The first miss does the backend call; concurrent requests for the same key wait on that one in-flight result. 5,000 requests become 1 backend call. The implementation holds a lock only around the table, never around the backend call, so unrelated keys still proceed in parallel.

```

# WHAT: request coalescing (singleflight), the fix for cache stampedes.
# WHY:  when a hot key expires, N concurrent requests must produce ONE
#        backend call, not N. The first request does the work; the rest
#        wait on the same in-flight result.
# WHAT BREAKS: holding the lock while calling the backend serializes
#               everything. The lock must guard the TABLE only; the backend
#               call happens outside it.

import threading
import time

class Singleflight:
    def __init__(self):
        self._lock = threading.Lock()
        self._inflight = {} # key -> (event, result slot)
        self.backend_calls = 0

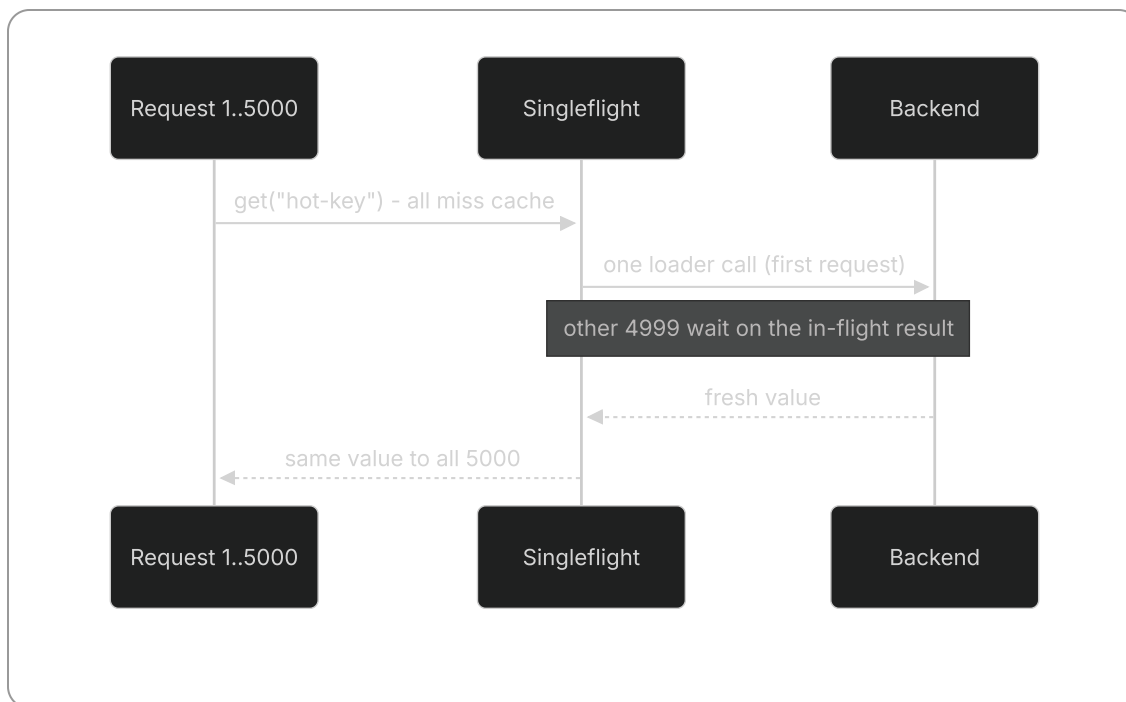
    def get(self, key, loader):
        with self._lock:
            if key in self._inflight:
                ev, slot = self._inflight[key]
                waiter = True
            else:
                ev, slot = threading.Event(), {}
                self._inflight[key] = (ev, slot)
                waiter = False

        if waiter:
            ev.wait() # share the in-flight result
            return slot["value"]

        try:
            self.backend_calls += 1
            time.sleep(0.05) # stand-in for a slow DB/model call
            slot["value"] = loader()
            return slot["value"]
        finally:
            with self._lock:
                del self._inflight[key]
            ev.set()

sf = Singleflight()
results = []
threads = [threading.Thread(
    target=lambda: results.append(sf.get("hot-key", lambda: "fresh-value"))
) for _ in range(50)]
[t.start() for t in threads]
[t.join() for t in threads]
print("requests:", len(results), "| backend calls:", sf.backend_calls,
      "| all equal:", len(set(results)) == 1)
# Verified output:
# requests: 50 | backend calls: 1 | all equal: True

```



Step 3: the other two fixes.

Jittered TTLs: spread expiry over a window so keys do not expire in lockstep (the 4 to 6 minute pattern from Section 1.5). **Probabilistic early refresh:** a background process refreshes hot keys before they expire; or each request refreshes with probability rising as expiry nears.

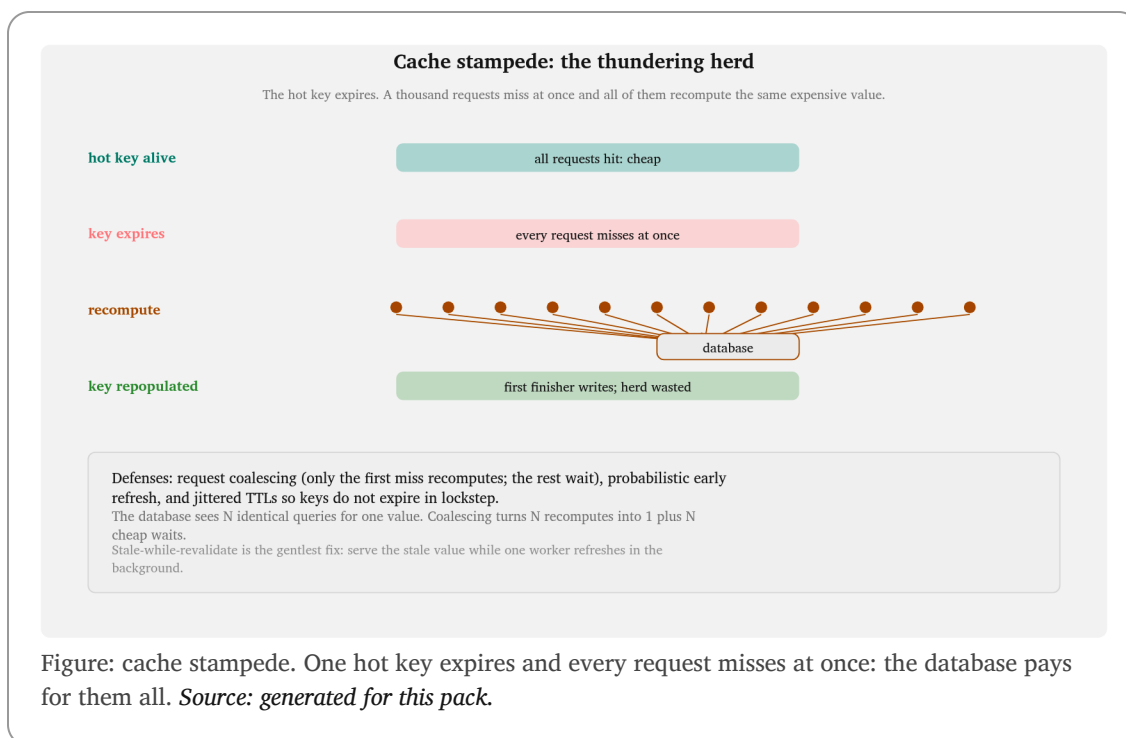


Figure: cache stampede. One hot key expires and every request misses at once: the database pays for them all. *Source: generated for this pack.*

How to read this diagram

1. While the hot key is alive, every request is cheap.
2. The key expires: every request misses at once and the database takes the full load.
3. The first finisher repopulates the key; the rest of the herd's work went to waste.
4. The fixes: cheap waits (request coalescing) or background refresh before expiry.

3.4 Confusing p99 with worst case**Step 1: what p99 means.**

p99 = 300 ms does not mean "the slowest request takes 300 ms." It means 1 percent of requests are slower than 300 ms. At 10,000 requests per second, that is 100 slow requests every second, each one a user having a bad time. The trap: reporting the p99 as if it describes the tail. It describes where the tail starts.

Step 2: tails multiply across hops.

A request that fans out to 20 backend calls, each with $p99 = 100$ ms, is not $p99 = 100$ ms end to end. The chance that all 20 calls land under their p99 is $0.99^{20} \approx 0.818$. So about 18 percent of end-to-end requests exceed 100 ms somewhere. The end-to-end p99 is set by the worst hop, and every hop you add drags it up.

Tails multiply across hops

20 hops, each $p99 = 100$ ms

$P(\text{all 20 under p99}) = 0.99^{20} = 0.818$

18% of end-to-end requests exceed 100 ms somewhere

The end-to-end p99 is set by the worst hop.

p99 is where the tail starts, not the worst case.

Figure: tails multiply. 20 hops at $p99 = 100$ ms each: $0.99^{20} = 0.818$, so 18% of end-to-end requests exceed 100 ms somewhere.

The tail rule. p99 is where the tail starts, not the worst case. Tails multiply across hops: 20 hops at p99 each means about 18% of end-to-end requests exceed 100 ms somewhere. Report the p99 as the point the tail starts, never as the slowest request.

TAIL COMPOSITION (20 hops, each p99 = 100 ms)

$P(\text{all 20 under 100 ms}) = 0.99^{20} = 0.818$

$P(\text{at least one over}) = 1 - 0.818 = 0.182$

End-to-end p99 >> any single hop's p99.

Rule: budget the tail per hop, measure p99.9/p99.99 for anything user-facing.

Step 3: the rules.

Measure p99.9 and p99.99 for user-facing paths. Keep fan-out shallow, or budget each hop's tail explicitly. When someone quotes a p99, ask "at what request rate," because 1 percent of a big number is a big number.

p99 vs max: the tail you can see and the tail you cannot

p99 ignores the worst 1%. Max is whatever the single worst request did. Both lie in different ways.



p99 hides the worst 1%: at 10k requests/s that is 100 users per second with an experience you never measured.

Max is dominated by one outlier: a single GC pause becomes the headline number. Report p99 plus the max, and read them together.

Tail at scale: a request touching 100 services at p99 each succeeds end-to-end only 37% of the time (0.99^{100}).

Figure: p99 vs max. p99 is the tail you can see; the max is the single worst request, the tail you cannot budget from. *Source: generated for this pack.*

How to read this diagram

1. p99 is the tail you can see: 99 percent of requests are faster.
2. The max is the single worst request: rare, real, and not plannable.
3. Budget from p99; investigate the max. Confusing the two is the trap.

3.5 Silent queue backlogs

Step 1: the boring outage.

A queue grows 2 percent faster than the consumer drains it. Nothing alerts. Latency creeps up over days. Then a deploy restarts the consumer. The team must reprocess the backlog, and the queue that looked "a little behind" turns out to hold 14 hours of work.

Step 2: depth is meaningless; age is everything.

The trap is monitoring queue depth instead of queue age. Depth without a rate is meaningless: 100,000 messages is 10 seconds at 10k/s and 28 hours at 1/s. Monitor lag in seconds:

The queue rule. Monitor queue lag in seconds and its growth rate, not raw depth. 100,000 messages is 10 seconds at 10k/s and 28 hours at 1/s. Depth without a rate is meaningless.

```
lag_seconds = queue_depth / consumption_rate_per_s
```

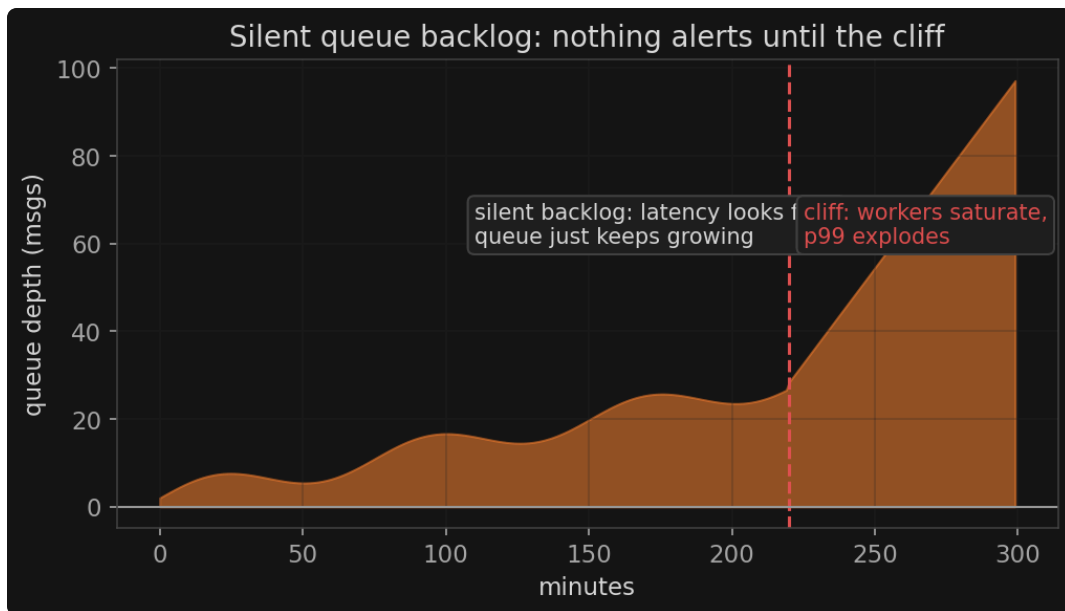


Figure: a queue that grows silently for hours while latency looks fine, then hits a cliff when workers saturate. Monitor queue lag in seconds and its growth rate, not raw depth. Alert on DLQ age.

Source: generated for this pack.

Step 3: alert on lag and its growth.

Alert on two things: lag exceeding a fraction of the freshness contract (say, 10 percent of the allowed staleness), and lag growing over a sustained window even while below the threshold. The growth alert catches the 2-percent drift months before it becomes an inci-

dent. The same trap hides in dead-letter queues: a DLQ with no reader and no alert is a second silent backlog. Every DLQ needs a reader (even a daily digest to a human) and an age alert.

Key takeaways

- Fencing tokens make split-brain harmless. Election alone does not.
- Exactly-once delivery is impossible; effectively-once (at-least-once + idempotent receiver) is the real target.
- Stampedes die to singleflight coalescing, jittered TTLs, and early refresh.
- p99 is where the tail starts, not the worst case; tails multiply across hops.
- Monitor queue lag in seconds and its growth rate, not raw depth. Alert on DLQ age.

Next: Part 4 tests the material: 26 rapid-fire drills with worked answers.

Part 4. Rapid-fire drills

This part is the checkpoint: 26 drills with worked answers. Cover the answers, answer out loud, then reveal. The categories match the platform: distributed systems, APIs and queues, storage, observability, pipelines, ML literacy, capacity planning.

Distributed systems

D1. A metadata store runs 5 replicas with $W = 3$, $R = 3$. Two replicas go down during a deploy. Do reads and writes still work? What breaks if a third replica fails?

Answer: reads and writes both still work, but there is no failure tolerance left.

Worked: $W = 3$ needs 3 live replicas for a write; $R = 3$ needs 3 for a read. With 2 down, exactly 3 remain, so both succeed. The quorum rule holds ($3 + 3 > 5$), so reads still see the latest writes. But one more failure leaves 2 live replicas: both reads and writes block until a replica returns. The design survives 2 failures only while all 5 are healthy; during the deploy it survives zero.

D2. A network partition splits your 3-replica store ($W = 2$, $R = 2$) into a side with 2 replicas and a side with 1. Which side keeps serving writes, and what must the other side do?

Answer: the 2-replica side keeps serving; the 1-replica side must refuse writes (and, strictly, reads that need the quorum).

Worked: a write needs $W = 2$ acknowledgments. Only the majority side has 2 live replicas, so only it can complete writes. The minority side with 1 replica cannot reach quorum. If it accepted writes anyway, the two sides would diverge and the quorum guarantee would break. This is the availability half of the tradeoff: the minority side chooses correctness over availability.

D3. The partition heals. The old leader on the minority side never learned that the cluster had deposed it. It kept issuing writes to the one follower it could reach. What mechanism makes those writes harmless?

Answer: fencing tokens.

Worked: every write carries a token from a monotonic sequencer. The new leader fetched token 42; the old leader still holds token 41. The store rejects any write whose token does not exceed the highest token seen, so the old leader's writes die on arrival. Without fencing, the healed cluster has two divergent histories and no quorum math can repair them.

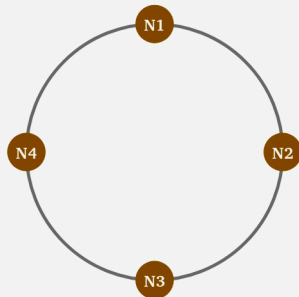
D4. You need a consensus group that tolerates 2 simultaneous failures. How many nodes, and how many votes to commit a write?

Answer: 5 nodes, 3 votes.

Worked: the rule is $2f + 1$ nodes to tolerate f failures, so $f = 2$ needs 5. A commit needs a majority: $\text{floor}(5/2) + 1 = 3$. With 2 nodes down, 3 remain, which is exactly a majority, so the cluster keeps committing. Note 4 nodes also tolerate only 1 failure (majority of 4 is 3, and $4 - 1 = 3$), so the even number buys nothing: always size consensus groups odd.

D5. 1M keys are spread over 10 nodes by hash mod 10. The operator removes one node. About how many keys move with consistent hashing versus naive mod-N hashing?

Consistent hashing: only K/n keys move



1M keys, 10 nodes, remove 1:

naive mod-10: ~all keys move

consistent hash: ~100k move

virtual nodes smooth the load

Figure: consistent hashing. With 1M keys over 10 nodes, removing one node moves ~all keys under naive mod-10 but only ~100k under consistent hashing.

Answer: about 100k with consistent hashing; nearly all 1M with naive mod-N.

Worked: consistent hashing places keys on a ring and only the removed node's keys ($K/N = 1M/10 = 100k$) need reassignment to its neighbor. Naive mod-N rennumbers every key's home (mod 9 instead of mod 10), so almost every key maps somewhere new. That is why membership changes use a ring, not a modulus.

APIs and queues

A1. A client POST to /jobs times out after the server already processed it. The client retries. Name the mechanism that stops a double charge on the customer, and the one detail that breaks it.

Answer: an idempotency key with a server-side key-to-result table; it breaks if the entry TTL is shorter than the retry window.

Worked: the client generates one key (say job-9f31) before the first attempt and reuses it on retries. The server runs the job once, stores key to result, and returns the stored result for repeats. If the TTL expires after, say, 1 hour but the client retries for 2 hours with backoff, the late duplicate finds no entry and re-executes. TTL must exceed attempts x backoff cap.

A2. One event stream must feed 6 consumers, and a new consumer next month needs the last 24 hours of history. Queue or stream, and why?

Answer: a stream (partitioned log).

Worked: a queue deletes each message on acknowledgment, so 6 consumers need 6 queues (or a pub/sub fan-out you now operate), and history is gone once consumed. A stream keeps the log durably; each consumer tracks its own offset, and the new consumer rewinds 24 hours and replays. One topic serves all current and future consumers.

A3. A consumer serves 100 requests per second. The latency deadline is 2 seconds. What queue bound do you set, and what happens to request 201?

Answer: bound 200; the server rejects request 201 fast

The queue bound: $100 \text{ req/s} \times 2 \text{ s} = 200$

request 1..200: queue

request 201: REJECT fast, 429 or 503

A queue past 200 means the 201st request waits past its deadline.

Queueing it wastes its time and the consumer's memory.

Rejecting fast lets the caller retry elsewhere or degrade gracefully.

Figure: the queue bound. 100 req/s with a 2 s deadline gives a bound of 200. The server rejects request 201 fast with 429 or 503. It never queues it slow.

with 429 or 503.

Worked: Little's Law, $\text{capacity} = \text{rate} \times \text{wait} = 100 \times 2 = 200$. A queue longer than 200 means the 201st request would wait more than its deadline even if everything ahead succeeds. Queueing it only wastes its time and the consumer's memory.

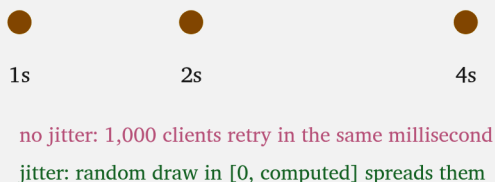
Rejecting fast lets the caller retry elsewhere, shed load, or degrade gracefully instead of queueing slow.

A4. A thousand clients hammer a recovering endpoint with retries in lockstep. Write the backoff formula that fixes it.

Answer: full jitter: $\text{sleep} = \text{random_between}(0, \min(\text{cap}, \text{base} \times 2^{\text{attempt}}))$.

Worked: exponential growth (1s, 2s, 4s, ...) spreads attempts over time. The random draw in $[0, \text{computed}]$ breaks the lockstep, so the thousand clients do not retry in the same millisecond and re-crash the endpoint. The cap (say 8 or 30 seconds) bounds the worst case. Missing jitter turns every recovery into a retry thundering herd.

Backoff spreads attempts; jitter breaks the lockstep



Cap the wait (say 8 or 30 s). Missing jitter = retry thundering herd.

Figure: backoff with jitter. Exponential growth (1 s, 2 s, 4 s) spreads attempts over time. The random draw breaks the lockstep so 1,000 clients do not retry in the same millisecond.

Storage

S1. A hot key's TTL expires and 5,000 requests per second hit the database at once. Name the failure and give two fixes.

Answer: a cache stampede (thundering herd). Fixes: request coalescing and jittered TTLs.

Worked: coalescing (singleflight) makes the 5,000 concurrent misses share one in-flight backend call, so the database sees 1 query instead of 5,000. Jittered TTLs (expire over 4 to 6 minutes instead of exactly 5) stop keys from expiring in lockstep, so the herd never forms. A third layer is probabilistic early refresh of hot keys before expiry.

S2. Your HNSW index scores $\text{recall@10} = 0.88$ against brute force. The target is 0.95 and the latency budget has headroom. Which knob do you turn, which direction, and when do you stop?

Answer: raise `ef_search` (say 50 to 100 to 200); stop at $\text{recall@k} \geq 0.95$ or when the latency budget runs out.

Worked: `ef` is the query-time beam width; wider beams examine more candidates, so recall rises and latency grows roughly linearly. Measure recall@10 against brute force on real queries after each step. If the latency budget runs out before 0.95, do not buy a bigger `ef`. Shard the index (split it across machines), quantize vectors, or accept the recall with a product-level fallback.

S3. Under mod-N hashing, one shard holds 40 percent of the keys while others sit idle. Give two fixes.

Answer: consistent hashing with virtual nodes, or a better shard key.

Worked: virtual nodes place many small virtual replicas of each physical node on the ring. That evens out the key distribution and smooths the load when nodes join or leave. If the skew comes from the key itself (say, shard by tenant and one tenant is huge), no hash fixes it. Split the hot tenant's key space or shard by a more uniform attribute.

S4. An HNSW index holds 1M vectors of dim 768 in FP32 with $M = 16$. Roughly how much memory?

Answer: about 3.2 GB total: 3.07 GB for vectors, ~128 MB for the graph.

Worked: vectors cost $1\text{M} \times 768 \times 4 \text{ bytes} = 3.07 \text{ GB}$. Graph edges cost about $1\text{M} \times 16 \times 2 \text{ (bidirectional)} \times 4 \text{ bytes} = 128 \text{ MB}$. The vectors dominate; graph overhead is under 5 percent. This is why vector quantization (FP32 to INT8) pays more than graph tuning when memory is tight.

Observability

O1. Your service has a 99.9 percent SLO. Errors run at 1 percent for 5 minutes. What is the burn rate, and does it page or ticket?

Answer: burn 10x; it pages.

Worked: $\text{burn} = \text{observed} / \text{budgeted} = 1\% / 0.1\% = 10$. At 10x, the 30-day budget is gone in 3 days (30/10). The page tier is 10x over 5 minutes, so this pages. A 0.2 percent error rate would be burn 2x over an hour: ticket, look today, no page.

O2. "Alert me when p99 latency exceeds 2 seconds" pages you at 3am for a single slow request. Yet it misses a real degradation holding 30 percent of traffic at 1.9 seconds. What is wrong, and what replaces it?

Answer: threshold alerts ignore the error budget; replace with burn-rate alerts.

Worked: a single 3-second request trips the threshold but consumes a negligible slice of the monthly budget: a false alarm. Thirty percent of traffic at 1.9 seconds never trips it but burns budget fast: a missed real incident. Burn-rate alerting (10x over 5 min pages, 2x over 1 h tickets) ties urgency to budget consumption, so each alert carries its own severity.

O3. A request fans out to 20 backend calls, each with $p99 = 100$ ms. Roughly what fraction of end-to-end requests exceed 100 ms somewhere?

Answer: about 18 percent.

Worked: $P(\text{all 20 under } p99) = 0.99^{20} \approx 0.818$, so $P(\text{at least one over}) = 1 - 0.818 \approx 0.182$. The end-to-end p99 is set by the worst hop and sits well above any single hop's p99. Rule: budget the tail per hop, and measure $p99.9/p99.99$ on user-facing paths.

O4. A service handles 10,000 requests per second with $p99 = 300$ ms. How many requests per second are slower than 300 ms?

Answer: about 100 per second.

Worked: p99 means 1 percent of requests exceed 300 ms. 1 percent of 10,000 is 100. p99 is where the tail starts, not the worst case. Reporting "p99 is 300 ms" as if the slowest request takes 300 ms hides 100 slow requests every second.

Pipelines

P1. A nightly job scores 10M documents with a model. The output may be a day old. Batch, micro-batch, or streaming, and why?

Answer: batch.

Worked: the freshness contract ("yesterday is fine") is the loosest possible. Batch is the cheapest per event, the simplest to reason about, and the easiest to re-run when something goes wrong. Streaming would cost more per event and add late-data and ordering complexity for zero product benefit. Match the shape to the contract.

P2. A ranking feature must reflect user events from the last 5 minutes. Which pipeline shape?

Answer: streaming, or tight micro-batch if minutes of delay are acceptable.

Worked: the freshness contract is seconds, which rules out batch entirely. Micro-batch with 1-minute windows is usually the pragmatic pick: it reuses batch logic and hits the contract with margin. Choose true per-event streaming only if the product needs sub-minute reaction and someone will operate the complexity (late data, out-of-order events, exactly-once effects).

P3. A poison message crashes the worker on every retry, forever. What breaks the loop?

Answer: an attempt cap plus a dead-letter queue with a reader and an alert.

Worked: cap attempts (say 5, with jittered backoff between them). After the cap, route the message to the DLQ instead of requeueing

The DLQ is a queue with an owner, not a grave

- human-readable payload
- daily reader or digest
- age alert

Without these it becomes a silent backlog.

Data loss gets discovered during an audit.

Figure: the DLQ needs three things. A human-readable payload. A daily reader or digest. An age alert. Without them it becomes a silent backlog.

: the poison stops blocking the healthy messages behind it. The DLQ needs a human-readable payload, a daily reader or digest, and an age alert. Without them it becomes a silent backlog where the team discovers data loss during an audit.

ML literacy

M1. A 70B model has 80 layers and hidden size 8192, served in FP16 with standard multi-head attention. What is the KV cache per token, and per 4k-token request?

Answer: about 2.5 MB per token; about 10 GB per 4k request.

Worked: $2 \text{ (K and V)} \times 80 \text{ layers} \times 8192 \text{ hidden} \times 2 \text{ bytes} = 2,621,440 \text{ bytes} \approx 2.5 \text{ MB}$ per token. Times 4096 tokens $\approx 10.7 \text{ GB}$. That is before weights (140 GB for 70B in FP16), so one request's cache alone exceeds an 80 GB GPU's spare room: long contexts are what OOM serving fleets.

M2. An 8B model's forward pass costs about 16 GFLOP per token. On hardware delivering 300 TFLOPs effective, what is the compute-bound time per token, and why is real decoding slower?

Answer: about 53 microseconds; real decoding is memory-bound.

Worked: $16 \text{ GFLOP} / 300 \text{ TFLOP/s} \approx 53 \mu\text{s}$. But each token's decode step must stream all 16 GB of weights from GPU memory. Memory bandwidth (not FLOPs) is the binding constraint, so the FLOP math is a floor, not a promise. This is also why batching helps: one weight-streaming pass serves many tokens at once.

M3. 8B weights in FP16 take 16 GB. What do INT8 and INT4 give you, and what do you give up?

Answer: 8 GB and 4 GB; you give up accuracy margin.

Worked: $8\text{B} \times 1 \text{ byte} = 8 \text{ GB}$; $8\text{B} \times 0.5 \text{ bytes} = 4 \text{ GB}$. Each step down the bit ladder roughly halves memory and doubles memory-bound throughput. The cost is accuracy cliffs on some tasks. Version and evaluate the quantized artifact like any model change. Never treat it as free compression.

Capacity planning

C1. Requests arrive at 120 per second; each holds a GPU slot for 1.5 seconds. How many slots, and how many GPUs at batch 8?

Answer: 180 slots; 23 GPUs before headroom.

Worked: Little's Law, $L = 120 \times 1.5 = 180$ concurrent slots. At 8 slots per GPU (batch 8), $180 / 8 = 22.5$, so 23 GPUs. That is the steady-state average. Add headroom for bursts and a spare per failure domain, and size the batch from the latency/throughput knee, not from this division.

C2. A fleet runs at 90 percent utilization and p99 latency keeps climbing. Name two fixes and the number that justifies the first.

Answer: add capacity to reach ~70 percent, or shed load; the M/M/1 curve justifies the first.

Worked: at $\rho = 0.9$ the mean queue wait is 9x the service time; at $\rho = 0.7$ it is 2.3x. The fleet is only about 29 percent over-provisioned relative to the 0.7 target $((0.9 - 0.7)/0.7)$, but that capacity buys a 4x latency cut. When adding machines is slow, shed load instead (reject fast, degrade features) to pull ρ down immediately.

C3. Demand is 100k tokens/s. One GPU sustains 900 tokens/s measured. Target utilization 0.7, plus 2 spares. How many GPUs?

Answer: 161 GPUs.

Worked: usable per GPU = $900 \times 0.7 = 630$ tokens/s. $100,000 / 630 = 158.7$, ceil to 159, plus 2 spares = 161. Every input here carries the answer: if the 900 was a brochure number at a different batch size, the whole answer is fiction. Measure S on your fleet at the target batch and context mix.

Size from measured S, not the brochure

$900 \text{ tok/s} \times 0.7 \text{ util} = 630 \text{ tok/s usable per GPU}$

$100,000 / 630 = 158.7 \rightarrow 159 \text{ GPUs}$

+ 2 spares = 161 GPUs

If the 900 was a brochure number, the whole answer is fiction.

Measure S on your fleet at the target batch and context mix.

Figure: fleet sizing from measured S. 900 tok/s at 0.7 util gives 630 usable tok/s per GPU. $100,000 / 630 = 159 \text{ GPUs plus 2 spares} = 161$.

Key takeaways

- Say the number, then the unit, then the reason. "180 slots" beats "a lot of GPUs."
- Every drill above reduces to one formula or one rule from Parts 1 and 2. If an answer feels like memorization, trace it back to the formula.
- The traps in Part 3 are where drills become incidents. Re-read them after your first on-call week.

size for the burst, not the average (Section 3.5)

average demand: 500 req/s burst demand: 2,000 req/s

Little's Law at burst: $2,000 \times 0.4 \text{ s} = 800 \text{ slots}$

steady-state sizing would leave you 600 slots short at the worst moment.

Next: Part 5 compresses everything into a one-page cheat sheet.

Part 5. One-page cheat sheet: 30 numbers and rules

Everything above, compressed to one page: the 30 numbers and rules that matter most.

Keep it open during design work.

1. **$W + R > N$** . Every read overlaps every write. The quorum rule.
2. **$W, R \leq N - f$** to survive f failed replicas. Pick the smallest pair that satisfies both rules.
3. **3, 2, 2** is the default: consistent, survives 1 failure.
4. **Consensus needs $2f + 1$ nodes, $f + 1$ votes**. 5 nodes tolerate 2 failures; always odd.
5. **Consistent hashing moves K/N keys** on membership change, not all K .
6. **Fencing tokens** make split-brain harmless. Election alone does not.
7. **Weakest consistency that keeps the product correct**. Money and leadership need linearizable.
8. **$L = \lambda W$** . Little's Law, in jobs. The first number in every capacity review.
9. **capacity = rate x max wait**. Past the bound, reject fast (429/503), never queue slow.
10. **$M/M/1 \text{ wait} = \rho / (\mu(1-\rho))$** . At 0.5: 1x; 0.8: 4x; 0.9: 9x; 0.95: 19x service time.
11. **Target 60-75% utilization; 85% is an incident**. Real traffic is burstier than Poisson.
12. **Queue or stream?** Replay, fan-out, or per-key order → stream. Simple work distribution → queue.
13. **Batch, micro-batch, or streaming?** Ask how stale the output may be. Yesterday → batch; seconds → stream; between → micro-batch.
14. **Full jitter:** `sleep = random_between(0, min(cap, base x 2^attempt))`.
15. **Idempotency key TTL > attempts x backoff cap**, or late duplicates re-execute.

16. **Effectively-once** = **at-least-once** + **idempotent receiver**. Exactly-once delivery does not exist.
17. **Stampede fix order**: singleflight coalescing, jittered TTLs, probabilistic early refresh.
18. **p99 is where the tail starts**, not the worst case. 1% of 10k/s = 100 slow requests/s.
19. **Tails multiply across hops**: 20 hops at p99 = 100 ms → ~18% of requests exceed it somewhere.
20. **Lag in seconds** = **depth** / **drain rate**. Alert on lag and its growth, not raw depth.
21. **Every DLQ needs a reader and an age alert**, or it is a silent backlog.
22. **HNSW**: **M** ~ 16, **ef_construction** ~ 200, tune **ef_search** (50-200) until **recall@k** > = 0.95.
23. **Below ~100k vectors, brute force wins**: simpler, perfect recall.
24. **KV cache** = 2 x **layers** x **hidden** x **bytes per token**. 8B FP16 ≈ 0.5 MB/token.
25. **Long contexts OOM fleets**, not model size. Check the model card for grouped-query attention.
26. **2 FLOP per param per token**. 8B → 16 GFLOP/token; a compute floor, since decode is memory-bound.
27. **Quantization halves memory per step** (FP16 → INT8 → INT4); version and eval each artifact.
28. **GPUs** = **ceil(Q x (T_{in} + T_{out}) / (S x 0.7))** + **spares**. Measure S on your fleet.
29. **Burn 10x over 5 min pages** (budget gone in 3 days); **burn 2x over 1 h tickets** (gone in 15 days).
30. **Nines per month**: 99% → 7.2 h; 99.9% → 43 min; 99.99% → 4.3 min; 99.999% → 26 s.

Last verified: September 2026. I executed every Python sample here during writing. Expected outputs sit in comments beside the prints that produce them. Quorum math ($W + R > N$), the M/M/1 wait formula, Little's Law, burn-rate alerting tiers, and the nines table are textbook results. HNSW's published structure (layered graphs, M, ef parameters) follows the original design; tuning ranges are standard practice. The KV-cache formula (2 x layers x hidden x bytes) holds for standard multi-head attention; grouped-query attention changes the constant. **UNVERIFIED**: the worked fleet numbers (GPU counts, tokens/s figures, request rates) are illustrative constructions for teaching the arithmetic, not measurements from a real deployment. H100 80 GB and FP16 throughput figures are representative 2026 hardware values, not vendor guarantees. Re-measure every input on your own fleet before sizing.

Where to go next. This is the last crash pack: the full crash sequence ends here. The natural next step is the [DSA Track: Top-300](#), the coding-interview companion to these systems packs. The unabridged version of everything here lives in the full track volumes; start from [the curriculum home](#).

\n

WATCH AND GO DEEPER

Two videos and four links that replay the pack's core ideas: consensus, queues, and the math that sizes an inference fleet.

Fast LLM Serving with vLLM and PagedAttention

The Berkeley authors walk through four things. KV cache fragmentation (20–40% utilization). The virtual-memory paging analogy. The block table. Copy-on-write for parallel sampling and beam search.

How LLM Inference Actually Scales: KV Cache, Batching & vLLM

Builds a modern serving engine piece by piece. Prefill vs decode. GQA/MQA. Continuous batching. PagedAttention. Prefix caching. Speculative decoding. Failure modes like KV exhaustion and requests stuck waiting behind a slow one. Per the video's own chapter list. This machine could not fetch the transcript.

GO DEEPER

Raft consensus

The Raft site: interactive visualizations and the paper behind the quorum math in Section 1.1.

Jepsen

Database consistency analyses: where the team stress-tests linearizability and exactly-once claims.

The PagedAttention paper (vLLM)

The full vLLM design behind Section 2.4's KV-cache math.

vLLM: Optimization and Tuning

The tuning guide: optimization levels O0–O3, attention backend selection, and the knobs that move the numbers in this pack.